

BILKENT UNIVERSITY

Department of Electrical and Electronics Engineering

Standalone Audio-Reactive LED Sequencer

A Pure-VHDL Audio-Reactive Lighting System on Basys 3

EE-102: Introduction to Digital Circuit Design

Term Project Report

Author: Can Kurc

Student ID: 22502100

Section: 02

Instructor: Prof. Ergin Atalar

Date: May 14, 2026

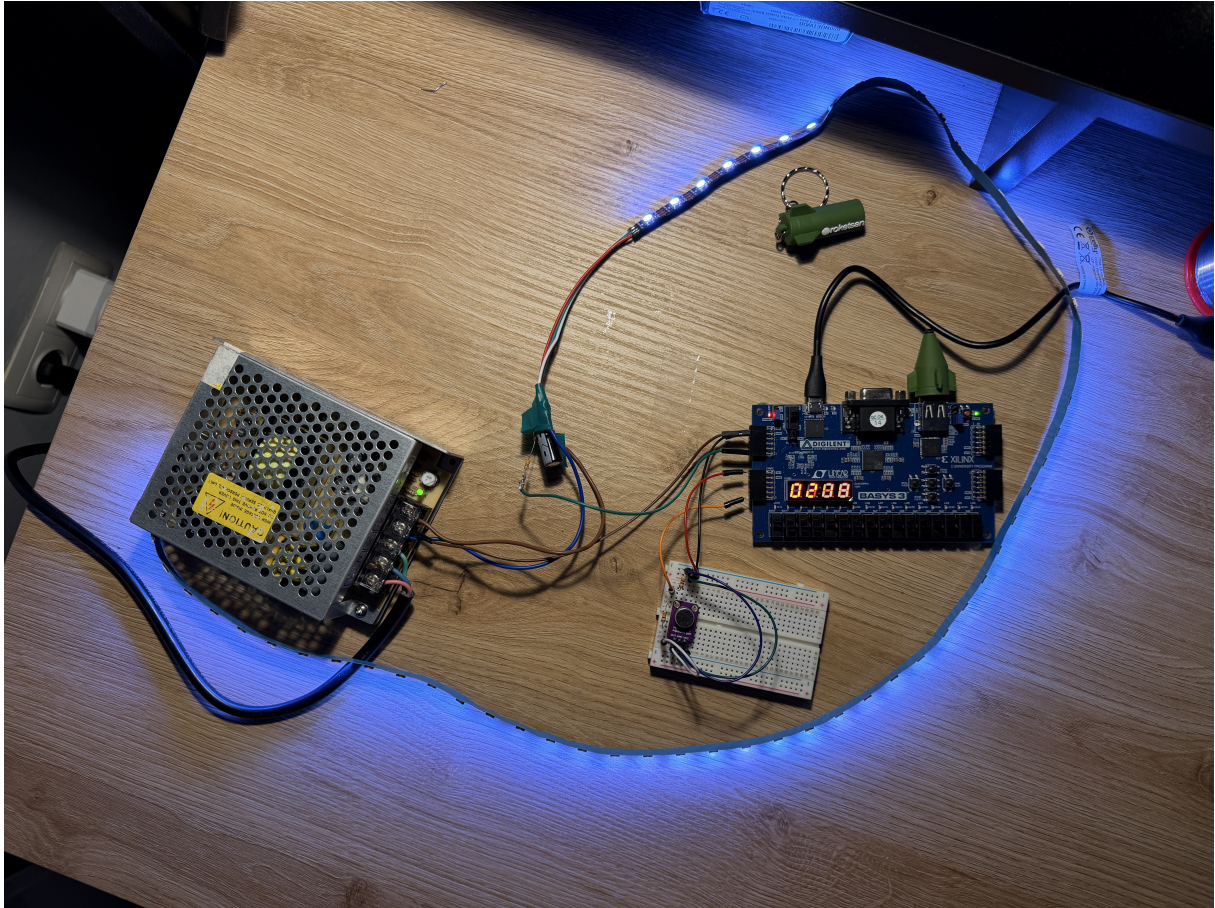


Figure: Final assembled system in operation.

Abstract

This report documents the design, implementation, and validation of a standalone, real-time audio-reactive LED sequencer built entirely in VHDL on a Digilent Basys 3 FPGA. The system listens to ambient audio through a MAX4466 electret microphone, samples it through the Artix-7 on-chip XADC at approximately 962 kSPS, filters and processes the digital stream through a moving-average filter and a hysteresis-based beat detector, and drives a 60-pixel WS2812B addressable LED strip with cycle-accurate, nanosecond-resolution timing. The complete pipeline is realised purely in synthesised digital logic, without any microcontroller, soft processor core, or software runtime. End-to-end latency from a detected beat to a visible color change is below 5 ms, and the LED strip refreshes at approximately 435 fps. The project was developed in two phases: Phase 1 implemented the WS2812B driver, color look-up table, and manual switch control; Phase 2 added the full audio-processing chain and a three-state animation FSM, swapped in with zero modification to Phase 1 modules thanks to a strictly modular architecture. The result is a deterministic, fully self-contained controller that outperforms commercial audio-reactive lighting products in both responsiveness and snap.

Contents

Abstract	1
1 Introduction	6
1.1 Project Overview	6
1.2 Motivation	6
1.3 Course Context	6
1.4 Scope and Organization of the Report	7
2 Design Specifications and Goals	7
2.1 Functional Requirements	7
2.2 Architecture Goals	7
2.2.1 Modular Design	7
2.2.2 Professional Code Style	7
2.3 Performance Targets	8
3 Hardware Construction	8
3.1 System Overview	8
3.2 Hardware Used	9
3.3 Microphone Front-End and Voltage Divider	9
3.4 Power Delivery for the LED Strip	10
3.5 Signal Integrity	11
3.6 Basys 3 I/O Pin Map	12
4 WS2812B LED Controller	12
4.1 The Challenge and WS2812B Protocol	12
4.2 FSM Architecture	13
4.3 The Counters	14
4.4 Color Latching and <code>frame_done</code> Signal	14
4.5 Timing Constants and Bit Extraction	15
4.6 FSM States and Logic	15
4.6.1 <code>RESET_STATE</code>	15
4.6.2 <code>SEND_STATE</code> Bit Generation	16
4.7 Color Look-Up Table and Brightness	17
5 Audio Processing — XADC and Moving Average	17
5.1 The XADC	17
5.2 Seven-Segment Display	18
5.3 Moving Average Filter	18
6 Hysteresis Beat Detection	19
6.1 Why Hysteresis	19
6.2 The Two-State FSM	20

7	Light Animation — beat_color_auto	21
7.1	The Three-State FSM	21
7.2	The Counters	22
7.3	The Triangle Wave	22
7.4	The Dual-Clock-Rate Design Pattern	23
7.5	<i>Nolurcalis</i> Story — Brightness Derivation	24
8	System Integration	24
8.1	led_subsystem Block Design	24
8.2	seven_segment_display Block Design	24
8.3	master_wrapper Top-Level Block Design	25
8.4	Constraints	25
8.5	Synthesis and Implementation Results	26
8.6	Complete System Data Flow	26
9	Testing and Validation	26
9.1	Test Methodology	26
9.2	Hardware Validation	26
9.3	Phase 1 Validation	27
9.4	Phase 2 Validation	27
9.5	Integrated System Tests	27
10	AI Usage Disclosure	28
10.1	The AI Approach	28
10.2	Specific AI Uses	28
10.2.1	Code-Style Conformance	28
10.2.2	Efficiency Optimizations	29
10.2.3	VHDL Concept Tutoring	29
10.2.4	Modular Code	29
10.2.5	The Report	30
10.2.6	Hardware Design	30
11	Conclusion	30
11.1	What was Built	30
11.2	What I Learned	30
11.3	Future Work, What Could Be Improved	31
	References	31
A	VHDL Source Listings	32
A.1	ws2812b_driver.vhd	32
A.2	color_lut.vhd	34
A.3	beat_color_auto.vhd	36
A.4	moving_average.vhd	39

A.5	hysteresis_detector.vhd	40
A.6	manual_color_select.vhd	42
A.7	binary_input_to_7_segment_display.vhd	42
A.8	anode_decoder.vhd	43
A.9	clock_divider.vhd	44
A.10	mux_4_to_1.vhd	44
B	XDC Constraints File	45
B.1	constraints.xdc	45
C	Vivado Block Design Screenshots	47
C.1	seven_segment_display Block Design	48
C.2	led_subsystem Block Design	48
C.3	master_wrapper Top-Level Block Design	49
D	Video Link	49

List of Figures

1	High-level block diagram of the complete system, showing the signal path from microphone, through voltage divider and FPGA processing, to the WS2812B LED strip.	9
2	Voltage divider and MAX4466 microphone module mounted on the breadboard, with jumpers leading to the Basys 3 JXADC port.	10
3	Mervesan MT-60-5 power supply with the 1000 μ F decoupling capacitor across V+ / V-, and jumpers to the LED strip and shared-ground line.	11
4	Two-state FSM driving the WS2812B strip: RESET_STATE latches the new frame, SEND_STATE serialises all 1,440 bits.	14
5	Vivado Block Design of the seven-segment display subsystem, showing the XADC Wizard IP, Slice block, Concat block, mux, BCD decoder, anode decoder, and clock divider.	18
6	Hysteresis band concept: the detector fires only when the signal crosses the upper threshold (high_th) and resets only when it falls below the lower threshold (low_th). Noise oscillation between the two thresholds is rejected.	20
7	Three-state animation FSM. A beat from any state instantly returns the system to FADING with the next color.	22
8	Vivado Block Design of led_subsystem : moving-average filter, hysteresis detector, beat_color_auto animation FSM, color_lut , and ws2812b_driver	24
9	Vivado Block Design of seven_segment_display : XADC Wizard, slice, concat, mux, BCD decoder, anode decoder, and clock divider.	25
10	Top-level master_wrapper block design instantiating both subsystems and connecting them to the Basys 3 external ports.	25
11	Vivado post-implementation utilization report for the Artix-7 XC7A35T target. The complete design consumes a small fraction of the available logic resources.	26
12	Integrated system demo: the LED strip cycles colors and fades in sync with a kick-drum-heavy track played through external speakers.	28
13	Detailed high-resolution export of the seven_segment_display Block Design.	48
14	Detailed high-resolution export of the led_subsystem Block Design.	48
15	Detailed high-resolution export of the master_wrapper Top-Level Block Design.	49

List of Tables

1	Basys 3 input and output pin mapping for the audio-reactive LED sequencer.	12
2	WS2812B single-wire NRZ bit-encoding timings used by the driver.	13
3	Example GRB color codes used in the look-up table (full brightness).	13

1 Introduction

1.1 Project Overview

Audio Reactive LED is a cycle-accurate VHDL synthesized project where everything happens in a Basys3 FPGA without the use of any microcontrollers, soft processor cores, or any software. This project listens to the audio in a room, isolates the bass beats, and lights up a 60-pixel LED strip in sync with the audio. The project demonstrates real-time digital signal processing and complex visual animation, lights fading between beats and breathing during silence, implemented entirely with combinational logic and finite-state machines.

Audio Reactive LEDs combine analog hardware design (voltage dividers, signal integrity, power delivery) with digital design in a single project. This pure hardware approach results in fully deterministic timing, which is essential for meeting the strict, nanosecond-accurate requirements of the WS2812B LED protocol. The whole system runs on Basys3's 100 MHz clock, samples audio at around 962 kSPS through the built in XADC and refreshes all 60 LEDs at around 435 frames per second. Every one of these numbers are fixed in hardware and cannot drift.

The project was built in two phases. Phase 1 implemented the LED protocol driver, the color look-up table, and manual control through the onboard switches. This was the milestone where the WS2812B timing and the analog front-end were first validated. Phase 2 replaced the switch reader with a beat detection pipeline (a moving average filter, hysteresis-based beat detector, and an animation FSM) without modifying any Phase 1 modules. The clean handover between phases is a result of the design philosophy described in Section 2.

1.2 Motivation

I was drawn to this project because it's a perfect intersection of my personal passion for music and lighting systems, and rigorous engineering. Challenging me to master advanced concepts such as cycle-level timing, hardware protocol synthesis and digital signal processing; this project also became a tool for me to learn and develop my engineering skills. It also gave me a lighting tool that I can use to decorate my room that would cost too much to buy.

Most audio-reactive lighting projects on the internet are built around a microcontroller running C code; the FPGA route is rarely taken because it forces you to think about every clock cycle individually. This is another reason why I chose this project, there is no library that hides the WS2812B protocol or a delay function, you must design everything yourself. The constraint is harsh but a great teacher.

1.3 Course Context

The project covers a large portion of the Bilkent EE-102 course syllabus, especially the weeks 10–12 material on finite state machine design methodology. The project is built on lab work, specifically Lab 2 with its hierarchical top modules with port maps, Lab 4 with logical shifts, Lab 5 with the implementation of the seven-segment display, multiplexer and clock divider circuits. It also extends these foundations, instead of a single FSM, the project orchestrates four FSMs and connects them into a complete system. The default-assignment pattern for FSM

outputs, dual-clock-rate synchronization between fast input pulse and a slow timing tick were concepts that went beyond the course material and learned during the project and are explained in their respective sections.

1.4 Scope and Organization of the Report

This report walks through the project starting from the analog front-end and building up to the integrated system. All VHDL source code, the Block Design files, and the constraints file are available in the project's GitHub repository at <https://github.com/CanKurc/EE102-Audio-Reactive-LEDs>. The most interesting code snippets are shown inline in the relevant sections of this report, the full source files appear in the appendices.

2 Design Specifications and Goals

2.1 Functional Requirements

Functionally, the system must be able to:

- Acquire analog audio from the microphone in real time.
- Smooth the audio to remove noise without losing beat envelope.
- Detect bass beats reliably without false re-triggering from signal jitter.
- Drive 60 LEDs at the strict WS2812B timing protocol (nanosecond-accurate).
- Cycle colors on every detected beat.
- Smoothly fade brightness between beats.
- Provide a breathing animation during prolonged silence.
- Allow live tuning of detection thresholds without re-synthesis.
- Recover to a known state via a hardware reset.
- Have a master on/off switch.

2.2 Architecture Goals

2.2.1 Modular Design

The project follows the **Single Responsibility Principle (SRP)** where every module has one job, one responsibility and one interface. Just like building lego, the user should be able to swap modules to change functions of the system without modifying any of the neighbors. With the use of swappable architecture, it becomes much easier to debug or update. This modular design resulted in a smooth transition between phase 1 and phase 2 where `manual_color_select` module (reads basys3 switches for color choosing and brightness) was replaced with `beat_color_auto` module (lights up the LEDs according to the sound signals) with zero modifications to any other modules like `color_lut` or `ws2812b_driver`.

2.2.2 Professional Code Style

Since the project covered an extended time period, some strict code styling principles were followed since most VHDL code becomes challenging to understand after some time. With the

use of these principles, the code was written in a way that it can be picked up by anyone with VHDL knowledge and be understood in minutes. This way, I was able to pick up my project from where I left off without the challenge of needing to remember what every line of my code does.

The code follows these principles:

- **KISS (Keep it Simple, Stupid):** Keep the code simple, make it easy to understand. Don't use complex, spaghetti algorithms if a simple if-else works. Keep it efficient.
- **Self-Documenting Code:** The code should be its own documentation. Use names that explain intent for any variables. Use constants instead of just plugging in the numbers (all timing values are declared as named constants at the top of the architecture).
- **Encapsulation:** Group data and behavior together. Each group hides its internal signals and exposes a clean interface.

AI was used to fit the code written into these strict architecture goals. More will be explained in Section 10.

2.3 Performance Targets

- End-to-end latency from acoustic event to LED color change: under 5 ms (imperceptible to human vision).
- LED frame rate: ≥ 430 fps to avoid visible flicker.
- Beat detection precision: one color change per kick drum hit, no doubles, no skipped beats with appropriate threshold tuning.
- Tunable threshold range covering quiet bedroom ($\sim 0x800$ baseline) to loud party environment ($\sim 0xC00+$ baseline).

The targets were chosen this way to build a reliable audio reactive controller that can fit any environment without any latency visible to the human eye. I wanted a perfect tool for myself since I am planning to use this project as a decoration for my room.

Section 3 begins the implementation walkthrough with the hardware construction that makes everything else possible.

3 Hardware Construction

3.1 System Overview

The system takes audio of the room using a MAX4466 microphone module, adjusts the voltage of the microphone to match the Basys3 XADC rates, sends the adjusted voltage to the FPGA. An external 5 V/12 A power supply connects to the LEDs with a 1000 μ F electrolytic capacitor across V+ and GND. A 330 Ω resistor series with the data cable of the LED to damp any corruption or reflection of the FPGA's outputs. This data cable connects to the FPGA.

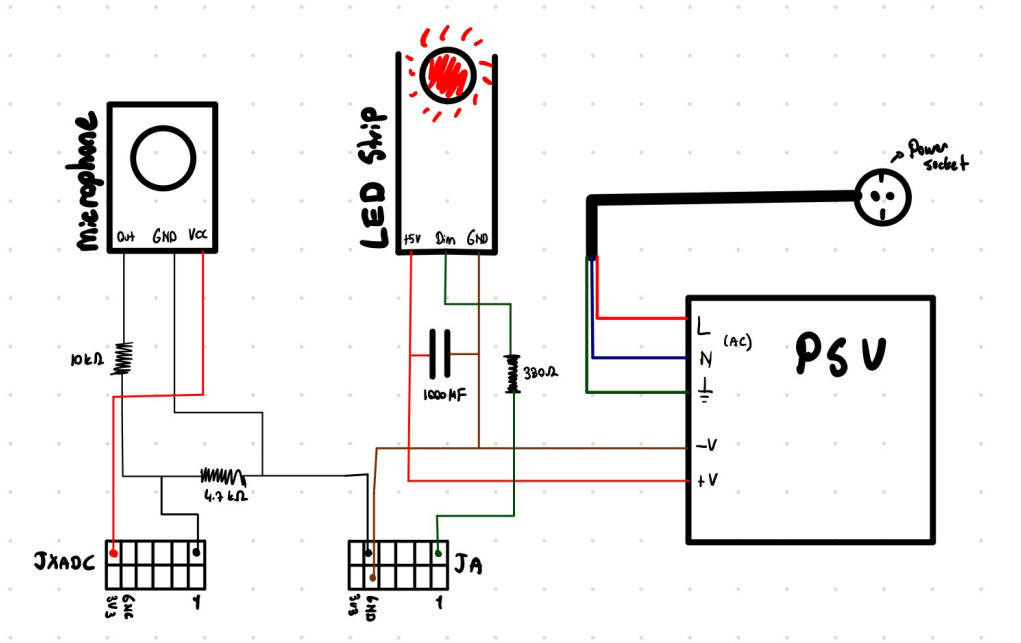


Figure 1: High-level block diagram of the complete system, showing the signal path from microphone, through voltage divider and FPGA processing, to the WS2812B LED strip.

3.2 Hardware Used

- Basys 3 FPGA (Artix-7 XC7A35TCPG236-1), 100 MHz internal clock.
- MAX4466 microphone module with adjustable gain.
- 10 kΩ / 4.7 kΩ voltage divider to step 3.3 V down to XADC-safe 1.05 V max.
- 60-pixel WS2812B addressable LED strip, GRB color order.
- Mervesan MT-60-5 5 V/12 A switching mode power supply for the strip.
- 330 Ω series resistor on the data line for transmission-line ringing protection.
- 1000 µF electrolytic decoupling capacitor at the LED injection point.
- All control via physical switches — no PC connection required at runtime.

3.3 Microphone Front-End and Voltage Divider

The MAX4466 microphone module outputs a 0 V to 3.3 V signal, biased around $V_{CC}/2 \approx 1.65$ V at idle. However, the Artix-7 XADC has a maximum rating of 1.0 V. Exceeding this can damage the analog front-end, so a voltage divider was required as protection. This voltage divider was designed to maximize signal-to-noise ratio while staying below the 1.0 V limit. The design was made with the help of AI, more to be explained in Section 10.

Mathematical Analysis

The transfer function of a two-resistor voltage divider is given by:

$$V_{\text{out}} = V_{\text{in}} \cdot \frac{R_2}{R_1 + R_2} \quad (1)$$

Substituting $R_1 = 10\text{ k}\Omega$ and $R_2 = 4.7\text{ k}\Omega$, the divider ratio becomes:

$$\frac{R_2}{R_1 + R_2} = \frac{4.7}{14.7} \approx 0.3197 \quad (2)$$

For the worst-case microphone rail of $V_{\text{in,peak}} = 3.3\text{ V}$:

$$V_{\text{out,peak}} = 3.3 \times 0.3197 \approx 1.05\text{ V} \quad (3)$$

This sits just barely below the 1.0 V XADC limit but is acceptable since the microphone rarely reaches the full rail. For the idle bias of $V_{\text{in,idle}} \approx 1.65\text{ V}$:

$$V_{\text{out,idle}} = 1.65 \times 0.3197 \approx 0.53\text{ V} \quad (4)$$

This matches the measured oscilloscope reading of 0.52 V , confirming the divider behaves as designed.

These resistor values were chosen since they provide the best ratio and voltage output to reduce signal-to-noise ratio and beat sensitivity. Resistors and the microphone were mounted on a breadboard.

Validation. The output voltage was measured using an oscilloscope. By blowing into the microphone, a pressure test was done and 1.05 V peak was validated.

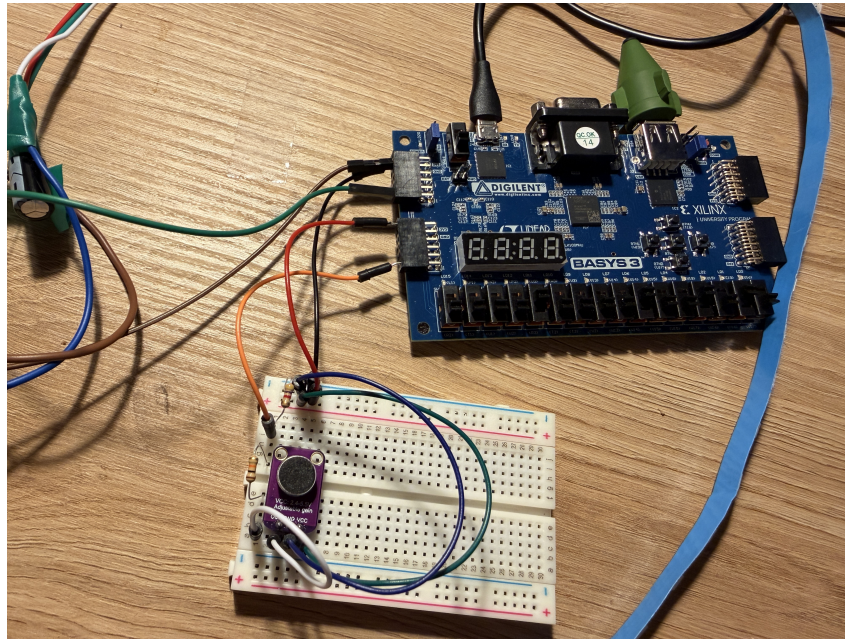


Figure 2: Voltage divider and MAX4466 microphone module mounted on the breadboard, with jumpers leading to the Basys 3 JXADC port.

3.4 Power Delivery for the LED Strip

Each WS2812B LED can draw up to 60 mA at a full white color so the theoretical maximum current draw is $60\text{ LEDs} \times 60\text{ mA} = 3.6\text{ A}$. The Basys3 5 V rail cannot supply this current, therefore an external power supply was required and Mervesan MT-60-5, $5\text{ V}/12\text{ A}$ rating gave

plenty of headroom. There were much better options to use but this power supply was already lying around in my house.

From a standard AC cable with ground connection, the tips of each smaller cable (live, neutral and ground connection) were stripped and connected to the power supply. The factory JST connection from the LED strip was removed for a direct connection with both the PSU and the FPGA. A $1000\mu\text{F}$ electrolytic capacitor was soldered across $V+$ and $V-$ outputs of the PSU to respond to the instantaneous current demand strike when the LEDs suddenly switch to full white. Without the capacitor, the strip's voltage may cause color glitches. A jumper was connected from the Mervesan's $V-$ terminal to PMOD JA Pin 5 (GND) of the Basys3 to create a shared ground between the PSU and the FPGA.

Validation. A continuity test across the PSU output terminals was done using a multimeter but initially showed a beep (apparent short circuit). After spending many hours checking every cable and making sure they're isolated, it was concluded that the multimeter was reading the SMPS's internal inductors and bleed resistors. Turns out SMPS internals look like shorts on a continuity test, and everything was working well. This test was a learning tool rather than a validation tool.

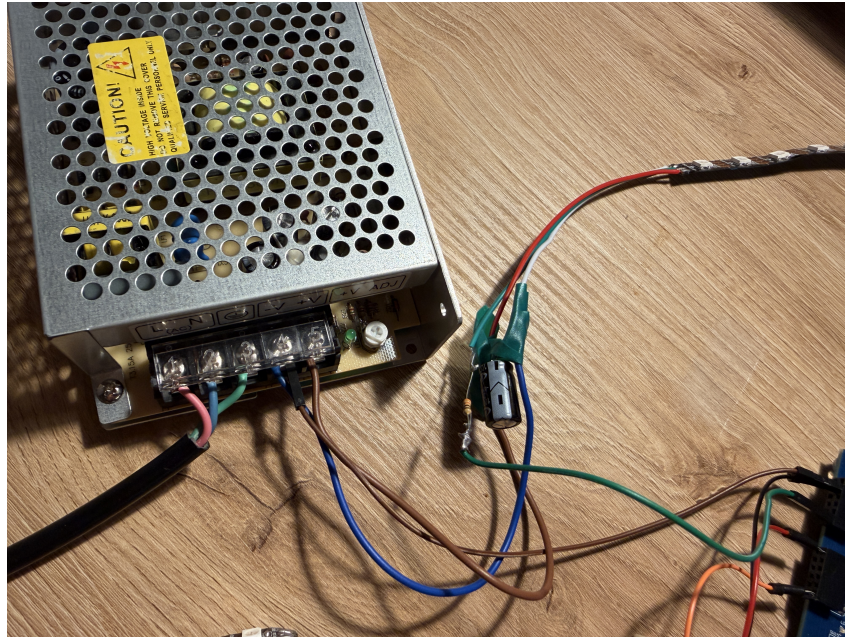


Figure 3: Mervesan MT-60-5 power supply with the $1000\mu\text{F}$ decoupling capacitor across $V+$ / $V-$, and jumpers to the LED strip and shared-ground line.

3.5 Signal Integrity

The Basys3's logic outputs very fast edges (nanoseconds) and steep rise times. At those speeds, even a short wire can behave like a transmission line and without termination, the data signal can overshoot, oscillate and corrupt the WS2812B's bit detection. A 330Ω resistor was soldered in series with the LED strip data cable, close to the first LED rather than the FPGA. 330Ω is low enough to not overwhelm the WS2812B's input impedance ($\sim 10\text{k}\Omega$ at the data pin) and high enough to provide series damping. It's also standard practice on Adafruit WS2812B guides [5, 6].

3.6 Basys 3 I/O Pin Map

Table 1 summarises the physical connections between the Basys 3 board and the external hardware.

Table 1: Basys 3 input and output pin mapping for the audio-reactive LED sequencer.

Signal	Basys 3 Pin	Direction	Connected To
Microphone analog in	JXADC Pin 1 (VAUX6)	Input	MAX4466 OUT via voltage divider
WS2812B data out	PMOD JA Pin 1	Output	Series $330\ \Omega \rightarrow$ LED strip DIN
Shared ground	PMOD JA Pin 5 (GND)	—	PSU V- and LED strip GND
Reset button	BTNC (centre)	Input	Hardware reset
Enable / on-off switch	SW15 (Pin R2)	Input	Master output enable
Lower threshold (6 bits)	SW6–SW11	Input	Hysteresis low threshold
Upper threshold (6 bits)	SW0–SW5	Input	Hysteresis high threshold
Seven-segment display	On-board pins	Output	Real-time XADC value (hex)
Clock	W5 (100 MHz)	Input	System reference clock

4 WS2812B LED Controller

4.1 The Challenge and WS2812B Protocol

The WS2812B is an integrated controller chip rather than a simple LED. Each chip contains a red, green, and a blue LED in addition to a small IC that listens to a serial data stream and PWMs the three colors. The strip has 60 of these LEDs connected in a chain. Each LED has a Data In (DIN) pin and a Data Out (DOUT) pin. The DOUT of LED #1 connects to the DIN of LED #2, and so on down the line. When data starts to be sent, LED #1 eats the first 24 bits it receives which are its GRB color. After that, it becomes a relay and passes every subsequent bit downstream to LED #2 and so on. To control all 60 LEDs, 1440 bits needs to be sent in one continuous stream.

The protocol of WS2812B is a single-wire NRZ with nanosecond timing tolerances. There is no clock line and the duration of each HIGH pulse encodes whether the bit is 1 or 0. This means the FPGA must count clock ticks with absolute precision, a single missed tick can cause corrupted color data for all 60 LEDs. The color data is in GRB (green, red, blue format), 24 bits per LED, 1440 bits total. After sending all 1440 bits for LED colors, a reset signal of a $\geq 300\ \mu\text{s}$ LOW needs to be sent. $500\ \mu\text{s}$ was chosen for safety margin.

Table 2: WS2812B single-wire NRZ bit-encoding timings used by the driver.

Symbol	Description	Nominal	Clock cycles @ 100 MHz
T0H	“0” bit HIGH duration	~400 ns	40
T0L	“0” bit LOW duration	~850 ns	85
T1H	“1” bit HIGH duration	~800 ns	80
T1L	“1” bit LOW duration	~450 ns	45
RES	Latch / reset LOW	≥300 μs	50 000

Table 3: Example GRB color codes used in the look-up table (full brightness).

Color	G	R	B	Hex (GRB)
Red	0x00	0xFF	0x00	x"00FF00"
Green	0xFF	0x00	0x00	x"FF0000"
Blue	0x00	0x00	0xFF	x"0000FF"
Yellow	0xFF	0xFF	0x00	x"FFFF00"
Cyan	0xFF	0x00	0xFF	x"FF00FF"
White	0xFF	0xFF	0xFF	x"FFFFFF"

Each channel’s value (0–255) controls how bright that color component is. So x"FF0000" is full-brightness green (recall the GRB byte order), while x"800000" is half-brightness green (because $0x80 = 128$, which is half of 255). To dim the entire LED, all three channels should be reduced proportionally. For example, half-brightness white is x"7F7F7F" instead of x"FFFFFF". This was done by right-shifting each color with the level amount in the `color_lut`.

One full frame (updating all 60 LEDs) looks like this:

- **Reset period:** Hold line LOW for 500 μs. LEDs display their stored colors.
- **Bit 0:** Send first bit of LED #1’s green channel (MSB first).
- **Bits 1–23:** Remaining bits for LED #1 (green, then red, then blue).
- **Bits 24–47:** LED #2’s data.
- ...
- **Bits 1416–1439:** LED #60’s data.
- **Back to step 1.** The cycle repeats.

Total frame time: $500 \mu\text{s} + (1440 \times 1.25 \mu\text{s}) = 500 + 1800 = 2300 \mu\text{s} = 2.3 \text{ ms}$. That’s about 435 frames per second — far faster than the eye can see, so changes look instantaneous.

4.2 FSM Architecture

The driver is a two-state FSM:

- **RESET_STATE:** Holds the data line LOW for 50,000 ticks. During this time, the LED strip latches whatever was sent in the previous frame. At the end, the input `color_in` (color input) is sampled into `active_color` (color to light) and `frame_done` is pulsed.
- **SEND_STATE:** Transmits all 1,440 bits of the current frame using the encoded HIGH-/LOW pulses.

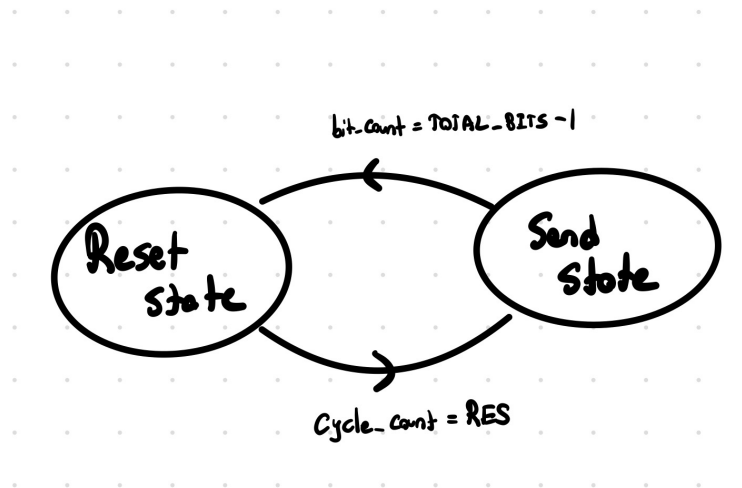


Figure 4: Two-state FSM driving the WS2812B strip: RESET_STATE latches the new frame, SEND_STATE serialises all 1,440 bits.

4.3 The Counters

The driver uses three independent counters working together. They work as a clock divider like in Lab 5 but uses integers instead of `STD_LOGIC_VECTOR` which makes comparison logic easier to read.

- **cycle_count:** Ticks within the current bit (0 to 65535). Counts clock cycles to time the HIGH/LOW phases of each bit.
- **bit_count:** Global position within the frame (0 to 1439). Determines when the frame is done.
- **bit_in_pixel:** Position within the current 24-bit pixel (0 to 23). Resets to 0 at each pixel boundary.

A separate counter was used to count bits in pixels instead of using `bit_count mod 24` because `mod 24` synthesizes to a hardware divider which is expensive in terms of hardware. Instead, this separate counter uses one comparator and one adder which is much cheaper.

4.4 Color Latching and frame_done Signal

The driver samples `color_in` into an internal `active_color` register once per frame during the RESET → SEND transition. SEND_STATE reads from `active_color` instead of `color_in` which means that if the upstream module changes `color_in` mid frame the driver doesn't see it until the next frame begins. Since the animation FSM (`beat_color_auto`) updates colors based on beats and frame ticks and the driver's `frame_done` signal, without this frame-boundary latching, LEDs 1–30 might show the old color while 31–60 shows the new one mid frame.

The driver outputs a `frame_done` signal that pulses HIGH for one clock cycle at the start of each new frame. This is the heartbeat for the entire animation system and the upstream modules such as `beat_color_auto` use it as a 435 Hz tick to drive its timing counters. This signal is the reason why the LED driver and the animation logic can use different timescales while remaining

synchronized.

4.5 Timing Constants and Bit Extraction

To fit the design specifications and make the code understandable, constants were used instead of just plugging in the numbers. This way, when a change needs to be made in a constant or when reading back through the code, it would be much easier. There are five constants for bit sending process.

```

1      constant TOH : integer := 40;      -- 400ns high for logic 0
2      constant TOL : integer := 85;      -- 850ns low  for logic 0
3      constant T1H : integer := 80;      -- 800ns high for logic 1
4      constant T1L : integer := 45;      -- 450ns low  for logic 1
5      constant RES : integer := 50000;   -- 500us reset/latch period

```

Listing 1: Timing constants used by `ws2812b_driver.vhd`.

There are also three constants that store information about our LEDs. This way, if we wanted to use different number of LEDs, it would be easy to make necessary changes since all you have to do is change a few constants.

```

1      constant NUM_LEDS      : integer := 60;
2      constant BITS_PER_LED : integer := 24;
3      constant TOTAL_BITS   : integer := NUM_LEDS * BITS_PER_LED;

```

Listing 2: Strip-geometry constants used by `ws2812b_driver.vhd`.

The WS2812B protocol requires data to be transmitted Most Significant Bit (MSB) first. The driver uses a dynamic index to do this efficiently without shifting the 24-bit `active_color` register on every cycle. By subtracting the `bit_in_pixel` counter from 23, the logic reverses the extraction order, and `current_bit` is received correctly. This is a concurrent assignment outside of the clocked FSM process. Since it's not gated by a clock edge, `current_bit` updates instantaneously when `bit_in_pixel` counter changes.

```

1      -- Extract current bit (MSB first within each 24-bit pixel)
2      current_bit <= active_color(23 - bit_in_pixel);
3
4      -- Output assignments
5      led_data_out <= internal_data;

```

Listing 3: Dynamic MSB-first bit extraction (concurrent assignment) in `ws2812b_driver.vhd`.

4.6 FSM States and Logic

4.6.1 RESET_STATE

This state holds the data line LOW until clock reaches `RES` cycles (50000 ticks). Then, it resets all the counters, adjusts the active color if it receives an enable input (turns the LEDs off if it doesn't) and transitions to `SEND_STATE`.

```

1      when RESET_STATE =>
2          internal_data <= '0';
3          if cycle_count < RES then
4              cycle_count <= cycle_count + 1;
5          else
6              cycle_count <= 0;
7              bit_count <= 0;
8              bit_in_pixel <= 0;
9
10         -- Latch the input color once per frame
11         if enable = '1' then
12             active_color <= color_in;
13         else
14             active_color <= (others => '0');
15         end if;
16
17         -- Signal upstream that a new frame is starting
18         frame_done <= '1';
19         state <= SEND_STATE;
20     end if;

```

Listing 4: RESET_STATE logic in ws2812b_driver.vhd.

4.6.2 SEND_STATE Bit Generation

For each bit, we need to generate the correct HIGH/LOW pulse. To achieve this, we first look at the current bit of `active_color`. If it's a 1, we drive high for T1H ticks and low for T1L ticks. If it's a 0, we drive high for T0H ticks and low for T0L ticks.

When the bit sending process is done, the `bit_count` is checked. If we reached the end of bits, it's transitioned to the RESET_STATE. Else, the counters get updated and the whole process repeats.

```

1      when SEND_STATE =>
2          if current_bit = '1' then
3              if cycle_count < T1H then
4                  internal_data <= '1';
5                  cycle_count <= cycle_count + 1;
6              elsif cycle_count < (T1H + T1L) then
7                  internal_data <= '0';
8                  cycle_count <= cycle_count + 1;
9              else
10                 cycle_count <= 0;
11                 if bit_count = TOTAL_BITS - 1 then
12                     state <= RESET_STATE;
13                 else
14                     bit_count <= bit_count + 1;
15                     if bit_in_pixel = 23 then
16                         bit_in_pixel <= 0;
17                     else
18                         bit_in_pixel <= bit_in_pixel + 1;
19                     end if;

```

```

20         end if;
21     end if;
22
23     else
24         if cycle_count < T0H then
25             internal_data <= '1';
26             cycle_count <= cycle_count + 1;
27         elsif cycle_count < (T0H + T0L) then
28             internal_data <= '0';
29             cycle_count <= cycle_count + 1;
30         else
31             cycle_count <= 0;
32             if bit_count = TOTAL_BITS - 1 then
33                 state <= RESET_STATE;
34             else
35                 bit_count <= bit_count + 1;
36                 if bit_in_pixel = 23 then
37                     bit_in_pixel <= 0;
38                 else
39                     bit_in_pixel <= bit_in_pixel + 1;
40                 end if;
41             end if;
42         end if;
43     end if;

```

Listing 5: SEND_STATE logic in ws2812b_driver.vhd.

4.7 Color Look-Up Table and Brightness

As part of the modular design, a color look-up table was built instead of plugging in the hex values every time. With the recommendations of AI (more in Section 10), a ROM was built to store all the color data.

Brightness scaling function works by right shifting each G, R, B value by the 3-bit **brightness_scale** value. Since right shifting is the same as dividing the values by 2, with every shift the brightness values are divided by 2. The complete implementation appears in the appendix and on GitHub.

5 Audio Processing — XADC and Moving Average

5.1 The XADC

The XADC is an Artix-7 FPGA tool that includes a 12-bit analog-to-digital converter. Its inputs range from 0 V to 1 V which is the reason why a voltage divider was necessary and outputs a 12-bit number (0–4095) representing the input voltage.

The XADC outputs 16 bits but the actual conversion result is 12 bits. The data is sitting in the upper 12 bits (15 to 4) and the lower 4 bits are just padding zeros. A slice IP block is used to get a clean 12-bit value.

The XADC is configured this way:

- Channel selection: VAUX6 (corresponds to JXADC Pin 1).

- Sequencer mode: Single channel, continuous.
- DRP (Dynamic Reconfiguration Port) interface for reading results.
- Self-triggering loop: `eoc_out` $\rightarrow$ `den_in` wired together so each conversion automatically triggers the next read.
- The DRP address for VAUX6 is `0x16` (decimal 22), driven by a constant block.

5.2 Seven-Segment Display

The seven-segment display design made in Lab 5 was copied to this project, showing the hex values of the 12-bit output of the XADC.

The data flow through the display module is:

- **Concat block** pads the 12-bit XADC data to 16 bits by adding 4 zero bits at the top.
- **mux_4to1** takes the 16-bit value and the 2-bit **digit_select** from the clock divider. Based on which digit is currently active, it selects the correct 4-bit nibble.
- **binary_input_to_7_segment_display** converts the selected 4-bit nibble to the 7 cathode signals that light up the correct segments (this is the Lab 5 BCD decoder).
- **anode_decoder** activates only one digit at a time based on **digit_select**.
- **clock_divider** cycles **digit_select** through 00→01→10→11→00... fast enough (~ 1 kHz) that all four digits appear to be lit simultaneously due to persistence of vision.

This is how my Lab 5 design worked; the only difference is that the input data comes from the XADC instead of switches.

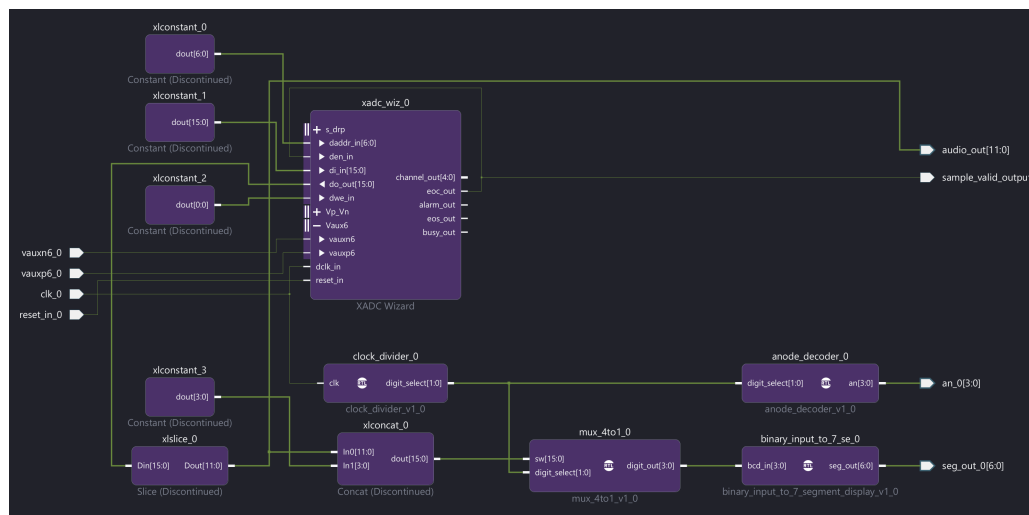


Figure 5: Vivado Block Design of the seven-segment display subsystem, showing the XADC Wizard IP, Slice block, Concat block, mux, BCD decoder, anode decoder, and clock divider.

5.3 Moving Average Filter

The XADC produces samples at 962 kSPS, and the raw stream is extremely noisy at the bit level. The least significant bits keep on constantly flipping due to noise, quantization and audio waveform structure. Beat detection from this signal might cause many false triggers from this noise. To reduce this noise, a low-pass filter can be made that smooths out the signal and

reveals the slow-moving energy envelope which represents the music dynamics. The simplest and cheapest low-pass filter is a moving average.

This filter works by taking the average of the last N samples, so a single noisy spike gets averaged out with the surrounding samples. The underlying slow trend is preserved because all samples shift up and down together. I chose N to be 4 because it's a balance between noise rejection and responsiveness and is also a power of 2 which makes the division far more easy.

A shift register was implemented to create an array of 4 samples. On every new sample from the XADC, the oldest sample is discarded, all the other samples shift up by one and the new sample enters at index 0. The sum of these 4 samples are computed and the sum is divided by 4 to get the average. Since 4 is a power of 2, division is possible by just reconnecting wires differently.

The 12 bits are resized to 14 before adding since $4 \times 4095 = 16380$ and 16380 requires 14 bits to represent. Without the resize, the additions might overflow and produce garbage. The value is then divided by 4 by shifting the most significant 12 bits right by 2 and discarding the last 2 bits which makes the output 12 bits again.

```

1      sum <= resize(samples(0), 14) +
2          resize(samples(1), 14) +
3          resize(samples(2), 14) +
4          resize(samples(3), 14);
5
6      audio_filtered <= std_logic_vector(sum(13 downto 2));

```

Listing 6: Moving-average filter core (moving_average.vhd).

6 Hysteresis Beat Detection

6.1 Why Hysteresis

A simple threshold solution where the code checks if the `audio_filtered` is greater than a threshold would fail since even after the moving average filter, the signal isn't still a clean square wave during the beat. So, in case the signal hovers over near the threshold, it crosses it back and forth dozens of times during a single beat. If this simple solution was used, this would generate a beat event, causing 10–20 false triggers and the LEDs would strobe chaotically instead of pulsing cleanly.

To eliminate this problem Hysteresis solution was used inspired by the Schmitt trigger [7] used in analog electronics. In this solution, the signal rises above one threshold to trigger the LEDs to light and falls below a lower threshold to switch off. The gap between them (the hysteresis band) is a dead space where the output doesn't change. This way, a noisy oscillation around a single threshold is prevented.

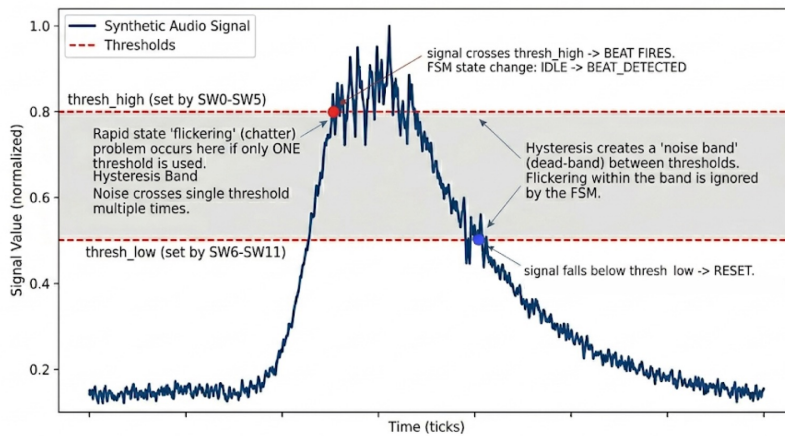


Figure 6: Hysteresis band concept: the detector fires only when the signal crosses the upper threshold (`high_th`) and resets only when it falls below the lower threshold (`low_th`). Noise oscillation between the two thresholds is rejected.

6.2 The Two-State FSM

A two-state FSM was made to build this hysteresis solution. The FSM-based approach is event-driven instead of being time windowed. So the detector waits for the signal to drop below the lower threshold rather than guessing how long a beat will last and possibly missing the next beat. The detector adapts to the length of each beat.

Since different rooms have different noise floors, hardcoded constants for the thresholds would mean one setting per bitstream and every threshold change would require re-synthesis. So the thresholds are connected to the physical switches of the Basys3 board making live threshold adjustments possible.

```

1  process(clk)
2  begin
3      if rising_edge(clk) then
4          if reset = '1' then
5              state      <= ARMED;
6              beat_pulse <= '0';
7
8          else
9              -- DEFAULT: beat_pulse is low every cycle unless explicitly
10             set.
11             -- This is what makes beat_pulse a single-cycle pulse instead
12             -- of a latching signal.
13             beat_pulse <= '0';
14
15             case state is
16
17                 when ARMED =>
18                     -- Waiting for a beat. Check if signal crossed above
19                     THRESH_HIGH.
20                     if unsigned(audio_filtered) > unsigned(thresh_high)

```

```

19         then
20             beat_pulse <= '1';
21             -- fire the pulse THIS cycle
22             state      <= FIRED;
23             -- lock out further beats
24         end if;
25
26         when FIRED =>
27             -- Beat already fired. Wait for signal to drop below
28             THRESH_LOW.
29             if unsigned(audio_filtered) < unsigned(thresh_low)
30                 then
31                     state <= ARMED;
32                     -- re-arm the detector
33                 end if;
34             end case;
35         end if;
36     end process;
37
38     beat_detected <= beat_pulse;

```

Listing 7: Two-state hysteresis beat detector (`hysteresis_detector.vhd`).

7 Light Animation — `beat_color_auto`

This module replaces the `manual_color_select` module from Phase 1 which reads the signals for color selection, brightness and enable from switches. This is the modular architecture paying off in concrete terms, instead of making challenging edits to the system, this design makes a switch possible by changing a few wires.

This module is responsible for the animations and visuals of the lights when a beat is detected or waited. Instead of lighting on and off in a boring way, this module creates a visually appealing animation that dances to the music. There are four visual phases to make the lights dance. When a beat is hit, a new color appears instantly in full brightness. Then it starts to fade with the strip's color smoothly fading to dark. When the fade completes, the strip starts waiting for 2.5 seconds. If no beat is detected in 2.5 seconds, a breathing animation occurs with the lights slowly increasing its brightness and later dimming in a triangular wave.

7.1 The Three-State FSM

- **FADING:** Entered on every beat. A brightness counter increments from 0 (bright) to 7 (dim) at a controlled rate.
- **FADED_WAIT:** Enters when the fade completes. In this state, the strip stays at brightness 7 (dim). An idle counter waits for `IDLE_LIMIT` frames before transitioning to the `BREATHING` state.
- **BREATHING:** After waiting for `IDLE_LIMIT` time, a triangular wave counter drives the

brightness up and down continuously, scaled down to the 8 brightness levels available.

(The `color_lut` module has a brightness input for 8 levels so the breathing levels are designed to fit this module.)

In any of the states if a beat gets detected, the state interrupts and returns to the FADING state with its new fully bright color. This is what makes the project responsive.

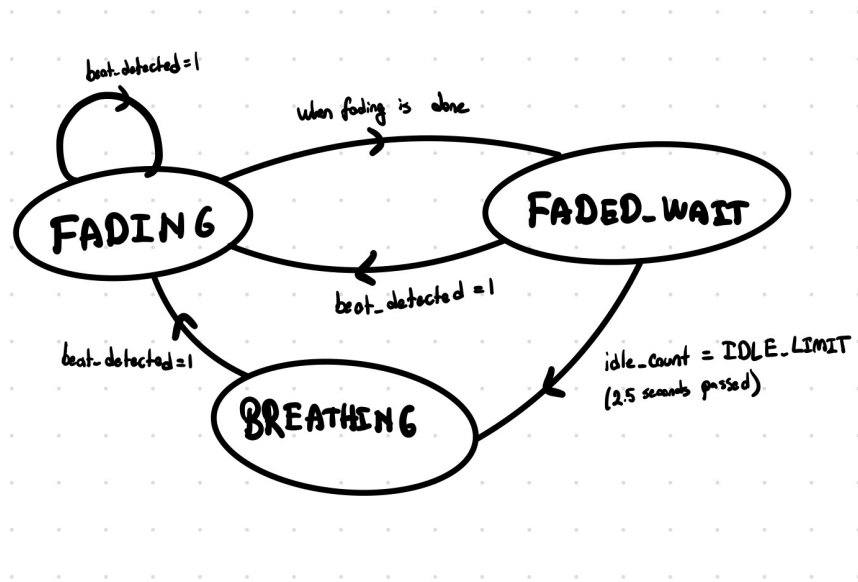


Figure 7: Three-state animation FSM. A beat from any state instantly returns the system to FADING with the next color.

7.2 The Counters

There are four different counters for this module, each with a specific role.

- **fade_count** (0 to $\text{FADE_RATE} - 1 = 0$ to 29): how many frames since the last brightness step during fading.
- **idle_count** (0 to $\text{IDLE_LIMIT} - 1 = 0$ to 1087): how many frames spent waiting at dim before breathing starts.
- **breath_tick** (0 to $\text{BREATH_RATE} - 1 = 0$ to 9): how many frames since the last breath counter update.
- **breath_count** (0 to 127): the triangle wave's position, drives the brightness mapping during breathing.

Each counter uses integer range 0 to N since Vivado synthesizes a register with the right width.

7.3 The Triangle Wave

For the breathing animation, AI recommended me to use a triangle wave since it's the simplest periodic function that produces smooth back-and-forth motion. A counter **breath_count** incrementing from 0 to 127 and then decrementing back from 127 to 0 was used to make this

triangle wave happen. A variable `breath_dir` was used to save the direction of the wave (0 means counting up, 1 means counting down) and a flip happens when `breath_count` hits 0 or 127. By extracting the first 3 bits of `breath_count`, a matching signal can be sent to the driver for the brightness value.

7.4 The Dual-Clock-Rate Design Pattern

This design solves a crucial problem in timing that may cause missed beats. The system has two different time scales involved, the 100 MHz clock and the `frame_done` signal from the WS2812B driver. Since the beat pulses are 10 ns wide (a single 100 MHz clock cycle from the hysteresis detector), gating everything to `frame_done` would result in almost all the beats missing. It's a really small possibility that the beat detection signal coincides with the `frame_done` signal which is 435 Hz. If everything were gated on the 100 MHz clock instead the timing counters would advance 230 000 times faster than intended and animations would complete in microseconds rather than the seconds human eye needs to see them.

The process runs on the 100 MHz system clock. In every clock cycle, a check for `beat_detected` is done which guarantees that the 10 ns pulse is captured. All the other counters are updated with the `frame_done` clock which is 435 Hz, so signals match the LED strip. This way, the animation responds to beats with zero visible latency and every signal matches the driver and the timing makes sense for the visual effect being designed.

```

1  process(clk)
2  begin
3      if rising_edge(clk) then
4          if reset = '1' then
5              state      <= FADING;
6              color_idx  <= 0;
7              nolurcalis <= 0;
8              idle_count <= 0;
9              fade_count <= 0;
10             breath_tick <= 0;
11             breath_count <= (others => '0');
12             breath_dir  <= '0';
13         else
14             -- beat always wins, check it first
15             if beat_detected = '1' then
16                 -- new color, full brightness, restart everything
17                 if color_idx = 15 then
18                     color_idx <= 0;
19                 else
20                     color_idx <= color_idx + 1;
21                 end if;
22                 nolurcalis <= 0;
23                 idle_count <= 0;
24                 fade_count <= 0;
25                 breath_tick <= 0;
26                 breath_count <= (others => '0');
27                 breath_dir  <= '0';
28                 state      <= FADING;

```

```

29         -- only update on frame ticks, not every 100 MHz cycle
30         elsif frame_done = '1' then
31             -- State machine logic here...
32         end if;
33     end if;
34 end if;
35 end process;

```

Listing 8: The Beat Priority Structure running on the 100MHz clock in `beat_color_auto.vhd`

7.5 *Nolurcalis* Story — Brightness Derivation

Although the design specification had a strict rule about naming, the name *nolurcalis* for an internal brightness register was well earned during an insidious debugging session where the LED breathing effect strobed violently instead of fading smoothly. The problem was a math error in the code. The bit-slice (`6 downto 3`) was extracting four bits (0–15) instead of the intended three. When counter was high, the equation became $7 - 14 = -7$ and since *nolurcalis* was declared with an integer range 0 to 7, this negative value caused weird behavior in the hardware, making the values wrap chaotically. The fix was just a change in a single number: (`6 downto 4`).

8 System Integration

All modules were brought together by block designs; `led_subsystem`, `seven_segment_display`, and `master_wrapper`. As the name suggests, `master_wrapper` instantiated both block designs and connected them. A nested structure because of the design specifications. The display block design owns the XADC module and the seven-segment display chain (since XADC and display share the clock and the audio source) while the LED subsystem owns the full DSP-to-LED chain.

8.1 `led_subsystem` Block Design

This subsystem contains the entire Phase 2 DSP chain in addition to the LED output.

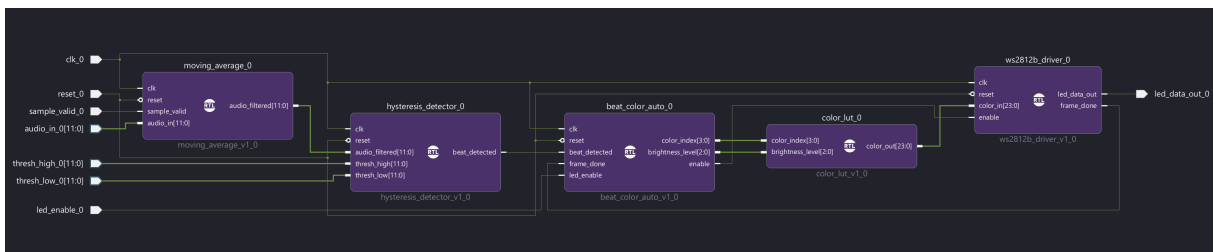


Figure 8: Vivado Block Design of `led_subsystem`: moving-average filter, hysteresis detector, `beat_color_auto` animation FSM, `color_lut`, and `ws2812b_driver`.

8.2 `seven_segment_display` Block Design

This subsystem contains the XADC Wizard IP and the seven-segment display modules.

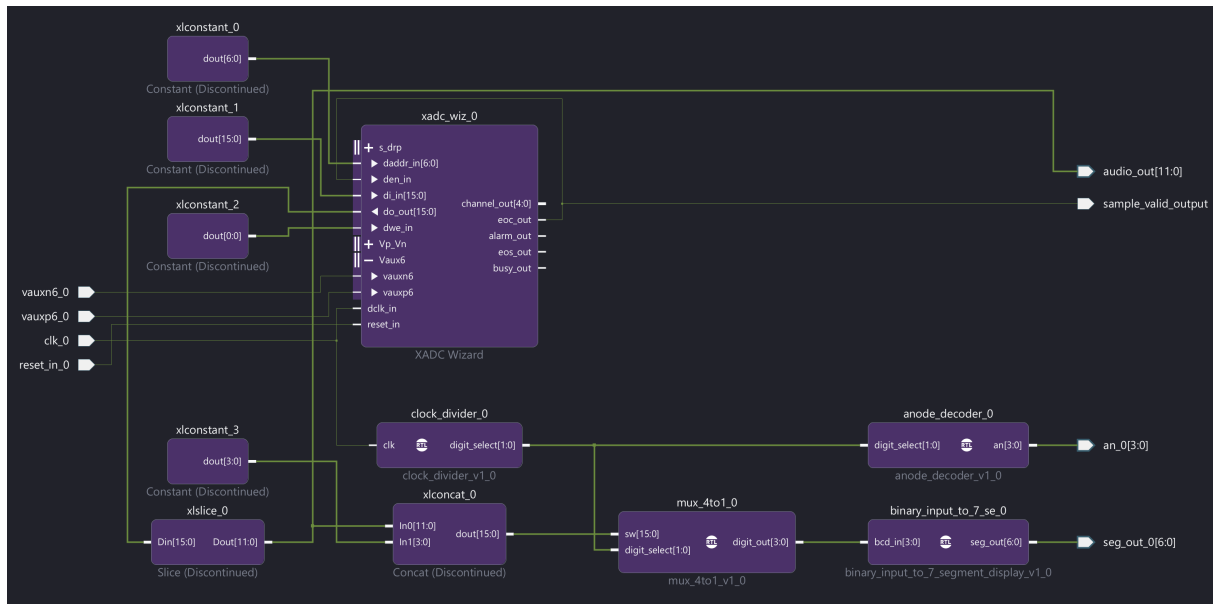


Figure 9: Vivado Block Design of `seven_segment_display`: XADC Wizard, slice, concat, mux, BCD decoder, anode decoder, and clock divider.

8.3 master_wrapper Top-Level Block Design

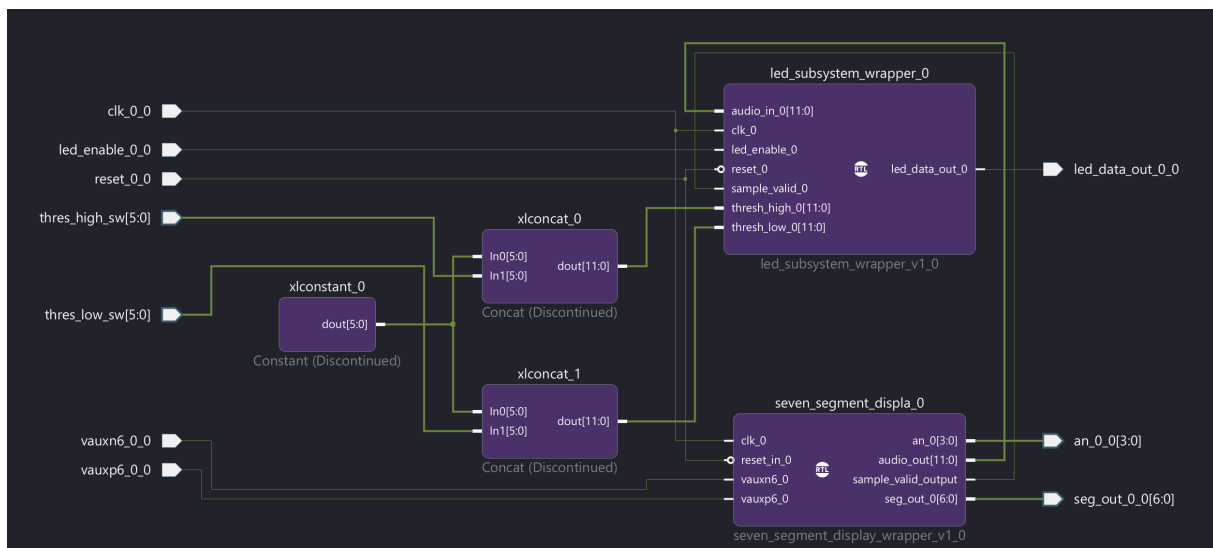


Figure 10: Top-level `master_wrapper` block design instantiating both subsystems and connecting them to the Basys 3 external ports.

8.4 Constraints

The constraints file (XDC) maps the Block Design external ports to the Basys3 pins and was written entirely by AI since the file only contains port information, more in Section 10.

The physical switches control the low and high thresholds in addition to the enable switch for the LEDs. The LED output is connected to the PMOD JA pin 1, and the microphone input is connected to the JXADC Pin 1. The center button is chosen as the reset button. The full constraints code is available in the GitHub repository and reproduced in Appendix B.

8.5 Synthesis and Implementation Results

The implementation utilization summary provided by Vivado proves the project's efficiency.

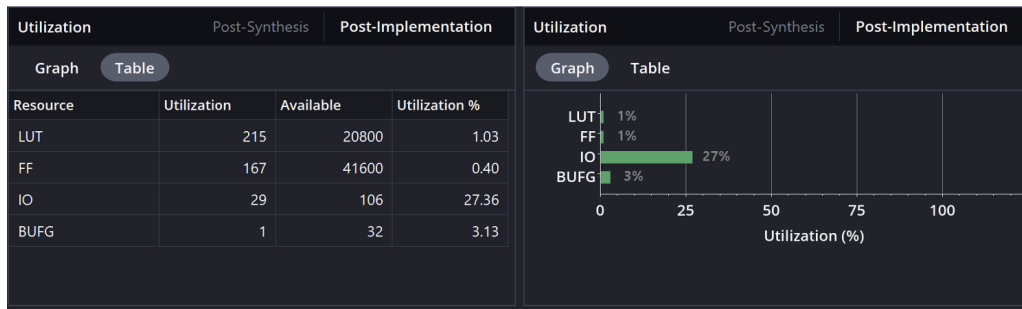


Figure 11: Vivado post-implementation utilization report for the Artix-7 XC7A35T target. The complete design consumes a small fraction of the available logic resources.

8.6 Complete System Data Flow

Sound waves are detected by the microphone which outputs an analog voltage data. The voltage is converted to the accepted values by the XADC and sent to the Basys3 by the JXADC Pin 1. The XADC produces a 16-bit value with the audio data being in bits 15:4. A slice IP extracts that 12-bit value and routes this data both to the seven-segment display chain and the moving average filter. The moving average outputs the average of the sound data to the hysteresis detector and this detector compares the filtered signal to the high and low thresholds. If the high threshold is crossed, a signal is sent to the `beat_color_auto` for one cycle long. `beat_color_auto` sees the pulse, cycles to the next color in the look-up table, resets the brightness to maximum and begins the fading state. The color look-up table takes the color index and looks up the 24-bit GRB value and applies the brightness scaling. `ws2812b_driver` receives this 24-bit color data, latches it into `active_color` on the next frame boundary and sends these 1440 bits over the data line using the PMOD JA pin 1. The strip then lights up the new color. The total latency from sound to light is under 5 ms and is imperceptible.

9 Testing and Validation

9.1 Test Methodology

To test if the system is working correctly, a layered approach was taken. It was confirmed that each piece was working in isolation before testing the combination. Static hardware tests were done first to make sure there were no faults hardware-wise. Isolated functional tests were done for Phase 1 and Phase 2 first before testing the integrated system. After integrated testing of the system, real-world scenarios were also tested with real music validation.

9.2 Hardware Validation

As explained in Section 3, separate tests were done for the voltage divider with oscilloscope readings and the power supply with a teaching short-circuit test and a load test with all 60 LEDs lighting white in full brightness. A test for shared ground was also made and confirmed that all

components shared the same ground.

9.3 Phase 1 Validation

A hardcoded `x"FFFFFF"` was driven to all 60 LEDs at full white to prove that the bit serialization, timing and daisy-chain systems worked well. Then a 16-color test was done, and all colors were verified with their switch combinations. An on-off switch test was done, and the reset button was tested which returns to color 0 at full brightness. The seven-segment display was already confirmed in Lab 5, so a microphone test was done directly and verified that the numbers fluctuated with sound. Phase 1 validation was also done by a TA and everything promised in the proposal was confirmed for Phase 1.

9.4 Phase 2 Validation

A moving average filter test was done to see if the least significant digit's flicker rate has reduced. It was confirmed that the moving average filter was working. A separate hysteresis detector test could not be done since the output is a 10 ns pulse. Instead, it was tested via the integration with the downstream LED behavior. After the completion of the `beat_color_auto`, manual tap test was carried out, and it was verified that the microphone produced one color change per tap when the thresholds were tuned correctly. Phase 2 was later confirmed by a TA and everything promised in the proposal was confirmed.

9.5 Integrated System Tests

First a manual tap test was done. With the strip on, a tap on the microphone resulted in a single-color change per tap which confirmed the full pipeline working. Then a music test was done with many genres. Since the audio processing is simple and the microphone quality is not ideal, the music had to be loud and sent through good speakers with good bass for good results. A phone speaker did not produce reliable results. With good speakers, a genre test was done with first checking the EDM genre. EDM music produced the best results with its clear 4-on-the-floor kick drum and consistent beats. Hip-Hop music also worked well since this genre also has strong bass. Rock music was later tested and it kind of worked with some false triggers on snares. Classical or ambient music was too dynamic and hard to tune, frequently triggering the breathing state. The results can be seen in the video provided.



Figure 12: Integrated system demo: the LED strip cycles colors and fades in sync with a kick-drum-heavy track played through external speakers.

10 AI Usage Disclosure

10.1 The AI Approach

For this challenging Basys3 project, AI was used but to get the best outcome out of this project (learning), AI was used as a tutor instead of a code generator. Just like I would do if I had a tutor 24/7 available to me, I asked AI to explain some logic and VHDL concepts that I didn't yet work on or understand then reimplemented the concept in my own code with my own structure. I did not paste AI-generated VHDL into this project unchanged without understanding. I did paste, however, the constraints directly from the AI since preparing constraints does not require any digital logic knowledge and I kept on changing the constraints for testing purposes.

10.2 Specific AI Uses

10.2.1 Code-Style Conformance

I had a clear vision for what a professional VHDL had to look like (see Section 2, design specifications) and I had AI check whether my code matched that vision and ask for any suggestions on improvements. For example, AI recommended me to use integer range types instead of unsigned where bit slicing wasn't needed which made my code much cleaner and easy to understand. I also asked AI to write or update comments for my code since some of them required too much unnecessary work (like the comments in the lookup table).

10.2.2 Efficiency Optimizations

Since I don't have much experience in designing circuits yet, in some cases I went with solutions that were too costly and hard to debug. For example, I initially designed a complex case-statement decoder for the color lookup table. AI later recommended me to use a ROM solution instead which is more concise and idiomatic for VHDL. I implemented the ROM version myself before learning about it in the class (that's why I was able to answer all questions correctly during the lecture).

10.2.3 VHDL Concept Tutoring

I first was designing the circuits on paper, then trying to transfer the equations to VHDL only writing equations to make things work. However, this approach started to fail since everything was getting more complicated as I moved on. It also was not fitting into my design specifications (Section 2) since after a few days, I wasn't understanding a single line in the code. With the help of AI and also our lectures, I was able to transition into writing code like I did on actual programming languages using necessary libraries. AI helped me get familiar with those libraries that made my VHDL program look like code that I was familiar with (like using integer values instead of unsigned). I asked AI to explain complex things that I didn't yet know about like the parallel semantics inside clocked processes, the default assignment pattern for FSM outputs, or why output ports can't be read in their own entity. I'd then apply the concepts I learned myself, asking AI to check my understanding by explaining them back to me.

Here is a practical example of how I used AI as a tutor and not a generator. I wanted to write the breathing triangle wave. I knew I needed a counter that goes up and down, but I wasn't sure how to express the direction flip clearly in VHDL. I asked the AI *"How to implement a triangle wave generator in VHDL using counters and direction flipping? Do not give me code but explain."* The AI explained the pattern conceptually saying to have a counter and a direction signal, incrementing when the direction is up and decremented when down. Told me to flip the direction signal when the counter hit any of the endpoints. I then wrote an example code myself, applying the things I've learned and asked the AI to check if I have done it right. If AI approves (just like a tutor), I integrated the code into the `beat_color_auto`'s BREATHING state and tuned the constants until the breathing looked smooth.

10.2.4 Modular Code

The first VHDL I was writing after learning new concepts were too mixed with all the functions repeating themselves. I had large chunks of code with everything included in them just to see somethings work but this mixture made debugging and progressing with new modules challenging for me. I asked AI to find and recommend encapsulations for all the functions, separating all the parts into as many modules as possible. I later checked all the recommendations and made necessary changes to the code. This made my debugging process much easier and helped me understand what the code is doing even after some time. Also, with those updates it was easy to continue the project from where I was left off after a midterm period.

10.2.5 The Report

I also used some AI writing this report as well. Since this project is long and this is my first time writing a detailed report for a project like this, I generated an outline with everything I should be talking about with the help of AI. I also made changes to some of my sentences to make them clearer and more professional using AI. I checked correctness and delivery of each section with an AI and got recommendations on some changes I should make. I did not write paragraphs from scratch or structure the report with an AI.

10.2.6 Hardware Design

Since I don't have much experience in hardware design, power delivery or voltage regulation I used AI to design safe hardware. I dealt with power right out of the socket and used AI as a safety inspector so I wouldn't face any serious problems with electricity and design the safest hardware possible. If I did not use AI for hardware design, I might have faced some serious trouble with power delivery. I assembled, soldered, wired and tested all the hardware myself, but designed everything with the help of AI, asking as many questions regarding safety as possible. AI helped me choose the right materials to use such as the power supply and explained how I should be assembling everything without any health risk.

11 Conclusion

11.1 What was Built

An audio reactive LED controller was built because of my passion for music and lighting systems. A basic analog microphone received the audio, sent it to the Basys3 FPGA by regulating the voltage. All the processing was done in the Basys3 as VHDL and Block Design, first filtering the audio, then using hysteresis to check if it passes the thresholds and making the LEDs dance to the rhythm of the music. The result was breathtaking, this controller worked better than the Philips Hue or Nanoleaf light systems I had at home. I was able to turn my passion into a tool for my learning and built myself a working controller that I was always dreaming of.

11.2 What I Learned

This project was a great introduction for me to the world of Digital Logic and taught me some great lessons for programming as well. I learned about the value of true modularity which paid off in many ways and saved me much time debugging or continuing the project after some time. I learned the importance of cycle level thinking in hardware design and the difference between something that “compiles fine” and “actually works on hardware” since undefined behaviors from out-of-range assignments don't get caught in compile time. Having to experience theoretical systems I've learned during lectures changed my vision and motivated me to further develop myself in the field.

11.3 Future Work, What Could Be Improved

Although the project works mostly fine, there are many areas where improvements can be made to this project. Better GRB values for the LEDs could be found for the color lookup table. A better microphone could be used to capture the audio better by filtering the noise hardware-wise. A more sophisticated DSP could be used with a frequency-domain beat detection with FFT. Multiple animation codes could be designed with selection between color wheels, ripple effects or frequency-selective lighting. A real PCB can be used to replace the breadboard wiring which would be more rugged and smaller. As I progress myself in this field, I am planning to apply what I've learned to develop this project and make this as a mirror of my development in the field.

References

- [1] S. Brown and Z. Vranesic, *Fundamentals of Digital Logic with VHDL Design*, 4th ed. New York: McGraw-Hill Education, 2023.
- [2] Digilent, Inc., *Basys 3 FPGA Board Reference Manual*. Digilent, 2014.
- [3] Advanced Micro Devices, Inc., *7 Series FPGAs and Zynq-7000 SoC XADC Dual 12-Bit 1 MSPS Analog-to-Digital Converter User Guide*, UG480. AMD, 2015. [Online]. Available: https://docs.amd.com/r/en-US/ug480_7Series_XADC/Instantiating-the-XADC
- [4] World Semi, *WS2812B Intelligent Control LED Datasheet*. Shenzhen: World Semi Co., Ltd.
- [5] Adafruit Industries, “Best Practices,” *Adafruit NeoPixel Überguide*, 2013. [Online]. Available: <https://learn.adafruit.com/adafruit-neopixel-uberguide/best-practices>
- [6] Adafruit Industries, “Powering NeoPixels,” *Adafruit NeoPixel Überguide*, 2013. [Online]. Available: <https://learn.adafruit.com/adafruit-neopixel-uberguide/powering-neopixels>
- [7] Wikipedia contributors, “Schmitt trigger,” *Wikipedia, The Free Encyclopedia*. [Online]. Available: https://en.wikipedia.org/wiki/Schmitt_trigger
- [8] C. Kurc, *EE102-Audio-Reactive-LEDs*, GitHub repository, 2026. [Online]. Available: <https://github.com/CanKurc/EE102-Audio-Reactive-LEDs>
- [9] C. Kurc, *Bilkent-EEE102-Labs*, GitHub repository, 2026. [Online]. Available: <https://github.com/CanKurc/Bilkent-EEE102-Labs>

Appendix A. VHDL Source Listings

The complete VHDL source for every module in this project is also available in the project's GitHub repository at <https://github.com/CanKurc/EE102-Audio-Reactive-LEDs>. The principal modules are included below in their entirety.

A.1 ws2812b_driver.vhd

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity ws2812b_driver is
6      Port (
7          clk          : in  STD_LOGIC;
8          reset        : in  STD_LOGIC;
9          color_in     : in  STD_LOGIC_VECTOR(23 downto 0);
10         enable       : in  STD_LOGIC;
11         led_data_out  : out STD_LOGIC;
12         frame_done    : out STD_LOGIC
13     );
14 end ws2812b_driver;
15
16 architecture Behavioral of ws2812b_driver is
17
18     -- WS2812B timing constants (100 MHz clock, 10ns per tick)
19     constant TOH : integer := 40;      -- 400ns high for logic 0
20     constant TOL : integer := 85;      -- 850ns low  for logic 0
21     constant T1H : integer := 80;      -- 800ns high for logic 1
22     constant T1L : integer := 45;      -- 450ns low  for logic 1
23     constant RES : integer := 50000;    -- 500us reset/latch period
24
25     constant NUM_LEDS      : integer := 60;
26     constant BITS_PER_LED : integer := 24;
27     constant TOTAL_BITS    : integer := NUM_LEDS * BITS_PER_LED;
28
29     -- FSM states
30     type state_type is (RESET_STATE, SEND_STATE);
31     signal state : state_type := RESET_STATE;
32
33     -- Counters
34     signal cycle_count : integer range 0 to 65535 := 0;
35     signal bit_count   : integer range 0 to TOTAL_BITS - 1 := 0;
36     signal bit_in_pixel : integer range 0 to 23 := 0;
37
38     -- Latched color for this frame (sampled once at frame boundary)
39     signal active_color : std_logic_vector(23 downto 0) := (others => '0');
40     signal current_bit  : std_logic;
41
42     -- Internal data line
43     signal internal_data : std_logic := '0';

```

```

44
45 begin
46
47     -- Extract current bit (MSB first within each 24-bit pixel)
48     current_bit <= active_color(23 - bit_in_pixel);
49
50     -- Output assignments
51     led_data_out <= internal_data;
52
53     process(clk)
54     begin
55         if rising_edge(clk) then
56             frame_done <= '0';
57
58             if reset = '1' then
59                 state <= RESET_STATE;
60                 cycle_count <= 0;
61                 bit_count <= 0;
62                 bit_in_pixel <= 0;
63                 internal_data <= '0';
64                 active_color <= (others => '0');
65
66             else
67                 case state is
68
69                     when RESET_STATE =>
70                         internal_data <= '0';
71                         if cycle_count < RES then
72                             cycle_count <= cycle_count + 1;
73                         else
74                             cycle_count <= 0;
75                             bit_count <= 0;
76                             bit_in_pixel <= 0;
77
78                             -- Latch the input color once per frame
79                             if enable = '1' then
80                                 active_color <= color_in;
81                             else
82                                 active_color <= (others => '0');
83                             end if;
84
85                             -- Signal upstream that a new frame is starting
86                             frame_done <= '1';
87                             state <= SEND_STATE;
88                         end if;
89
90                     when SEND_STATE =>
91                         if current_bit = '1' then
92                             if cycle_count < T1H then
93                                 internal_data <= '1';
94                                 cycle_count <= cycle_count + 1;
95                                 elsif cycle_count < (T1H + T1L) then

```

```

96         internal_data <= '0';
97         cycle_count   <= cycle_count + 1;
98     else
99         cycle_count <= 0;
100         if bit_count = TOTAL_BITS - 1 then
101             state <= RESET_STATE;
102         else
103             bit_count <= bit_count + 1;
104             if bit_in_pixel = 23 then
105                 bit_in_pixel <= 0;
106             else
107                 bit_in_pixel <= bit_in_pixel + 1;
108             end if;
109         end if;
110     end if;
111
112     else
113         if cycle_count < T0H then
114             internal_data <= '1';
115             cycle_count   <= cycle_count + 1;
116         elsif cycle_count < (T0H + TOL) then
117             internal_data <= '0';
118             cycle_count   <= cycle_count + 1;
119         else
120             cycle_count <= 0;
121             if bit_count = TOTAL_BITS - 1 then
122                 state <= RESET_STATE;
123             else
124                 bit_count <= bit_count + 1;
125                 if bit_in_pixel = 23 then
126                     bit_in_pixel <= 0;
127                 else
128                     bit_in_pixel <= bit_in_pixel + 1;
129                 end if;
130             end if;
131         end if;
132     end if;
133
134     end case;
135 end if;
136 end if;
137 end process;
138
139 end Behavioral;

```

Listing 9: ws2812b_driver.vhd

A.2 color_lut.vhd

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;

```

```

3  use IEEE.NUMERIC_STD.ALL;
4
5  entity color_lut is
6      Port (
7          color_index      : in  STD_LOGIC_VECTOR(3 downto 0);
8          brightness_level : in  STD_LOGIC_VECTOR(2 downto 0);
9          color_out        : out STD_LOGIC_VECTOR(23 downto 0)
10     );
11 end color_lut;
12
13 architecture Behavioral of color_lut is
14
15     -- 16-entry color table (GRB order for WS2812B)
16     type color_lut_type is array (0 to 15) of std_logic_vector(23 downto 0);
17     constant COLORS : color_lut_type := (
18         x"FF0000", -- 0: Green
19         x"00FF00", -- 1: Red
20         x"0000FF", -- 2: Blue
21         x"FFFF00", -- 3: Yellow (G+R)
22         x"FF00FF", -- 4: Cyan (G+B)
23         x"00FFFF", -- 5: Magenta (R+B)
24         x"FFFFFF", -- 6: White
25         x"80FF00", -- 7: Orange (needs more red)
26         x"FF8000", -- 8: Spring green (too less difference with the normal
27                     green)
28         x"0080FF", -- 9: Rose / pink (this is purple)
29         x"8000FF", -- 10: Teal (this is cyan)
30         x"00FF80", -- 11: Deep orange (this is pink)
31         x"FF0080", -- 12: Sea green (cyan)
32         x"4000FF", -- 13: Purple (blue)
33         x"20FF40", -- 14: Amber
34         x"606060"  -- 15: Warm white (dim) (just white)
35     );
36
37     -- Brightness scaling: right-shift each channel by the level amount
38     function scale_brightness(
39         color : std_logic_vector(23 downto 0);
40         level : std_logic_vector(2 downto 0)
41     ) return std_logic_vector is
42         variable g, r, b : unsigned(7 downto 0);
43         variable shift   : integer range 0 to 7;
44     begin
45         g := unsigned(color(23 downto 16));
46         r := unsigned(color(15 downto 8));
47         b := unsigned(color(7  downto 0));
48         shift := to_integer(unsigned(level));
49
50         g := shift_right(g, shift);
51         r := shift_right(r, shift);
52         b := shift_right(b, shift);
53
54         return std_logic_vector(g) & std_logic_vector(r) &

```

```

        std_logic_vector(b);
54     end function;
55
56 begin
57
58     -- Pure combinational: index into LUT, apply brightness, output result
59     color_out <= scale_brightness(
60         COLORS(to_integer(unsigned(color_index))),
61         brightness_level
62     );
63
64 end Behavioral;

```

Listing 10: color_lut.vhd

A.3 beat_color_auto.vhd

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  -- beat_color_auto: replaces manual_color_select for Phase 2
6  -- gets beat pulses in, sends color+brightness+enable out
7  -- also does the breathing thing when no beat for a while
8
9  entity beat_color_auto is
10     Port (
11         clk           : in  STD_LOGIC;
12         reset         : in  STD_LOGIC;
13         beat_detected  : in  STD_LOGIC;    -- from hysteresis
14         frame_done     : in  STD_LOGIC;    -- from ws2812b, ~435 Hz
15         led_enable     : in  STD_LOGIC;    -- physical switch, '1' turns
16         strip on
17         color_index    : out STD_LOGIC_VECTOR(3 downto 0);
18         brightness_level : out STD_LOGIC_VECTOR(2 downto 0);
19         enable         : out STD_LOGIC
20     );
21 end beat_color_auto;
22
23 architecture Behavioral of beat_color_auto is
24
25     type state_type is (FADING, FADED_WAIT, BREATHING);
26     signal state : state_type := FADING;
27
28     -- how many frames to sit fully dim before breathing starts
29     constant IDLE_LIMIT : integer := 1088;
30
31     -- how many frames between each dimming step
32     constant FADE_RATE  : integer := 30;
33
34     -- how many frames between each breath step

```

```

34     constant BREATH_RATE : integer := 10;
35
36     -- color picker, 0 to 15
37     signal color_idx      : integer range 0 to 15 := 0;
38
39     -- brightness, 0=full 7=dim
40     signal nolurcalis     : integer range 0 to 7  := 0;
41
42     -- counts frames spent sitting at fully dim (used in FADED_WAIT state)
43     signal idle_count     : integer range 0 to IDLE_LIMIT := 0;
44
45     -- counts frames between dimming steps, resets every FADE_RATE
46     signal fade_count     : integer range 0 to FADE_RATE  := 0;
47
48     -- counts frames between breath steps, resets every BREATH_RATE
49     signal breath_tick    : integer range 0 to BREATH_RATE := 0;
50
51     -- the breathing counter, 0 to 127 and back down
52     signal breath_count   : unsigned(6 downto 0) := (others => '0');
53
54     -- '0' breathing is up, '1' is down
55     signal breath_dir     : std_logic := '0';
56
57 begin
58
59     -- outputs
60     color_index           <= std_logic_vector(to_unsigned(color_idx, 4));
61     brightness_level     <= std_logic_vector(to_unsigned(nolurcalis, 3));
62     enable                <= led_enable;
63     -- wired straight to the switch
64
65     process(clk)
66     begin
67         if rising_edge(clk) then
68             if reset = '1' then
69                 state          <= FADING;
70                 color_idx      <= 0;
71                 nolurcalis     <= 0;
72                 idle_count     <= 0;
73                 fade_count     <= 0;
74                 breath_tick    <= 0;
75                 breath_count   <= (others => '0');
76                 breath_dir     <= '0';
77
78             else
79                 -- beat always wins, check it first
80                 if beat_detected = '1' then
81                     -- new color, full brightness, restart everything
82                     if color_idx = 15 then
83                         color_idx <= 0;
84                     else
85                         color_idx <= color_idx + 1;

```

```

86         end if;
87         nolurcalis    <= 0;
88         idle_count    <= 0;
89         fade_count    <= 0;
90         breath_tick   <= 0;
91         breath_count  <= (others => '0');
92         breath_dir    <= '0';
93         state         <= FADING;
94
95         -- only update on frame ticks, not every 100 MHz cycle
96     elsif frame_done = '1' then
97
98         case state is
99
100             when FADING =>
101                 if fade_count < FADE_RATE - 1 then
102                     fade_count <= fade_count + 1;
103                 else
104                     fade_count <= 0;
105                     if nolurcalis < 7 then
106                         nolurcalis <= nolurcalis + 1;
107                     else
108                         -- fully dim, time to sit and wait
109                         -- before breathing
110                         idle_count <= 0;
111                         state <= FADED_WAIT;
112                     end if;
113                 end if;
114
115             when FADED_WAIT =>
116                 -- sit here at nolurcalis=7 for IDLE_LIMIT frames
117                 if idle_count < IDLE_LIMIT then
118                     idle_count <= idle_count + 1;
119                 else
120                     -- waited long enough, start breathing
121                     breath_count <= (others => '0');
122                     breath_dir <= '0';
123                     breath_tick <= 0;
124                     state <= BREATHING;
125                 end if;
126
127             when BREATHING =>
128                 if breath_tick < BREATH_RATE - 1 then
129                     breath_tick <= breath_tick + 1;
130                 else
131                     breath_tick <= 0;
132
133                     -- triangle wave, goes up then down then up
134                     if breath_dir = '0' then
135                         if breath_count = 127 then
136                             breath_dir <= '1';
137                             -- flip to going down

```

```

137         else
138             breath_count <= breath_count + 1;
139         end if;
140     else
141         if breath_count = 0 then
142             breath_dir <= '0';
143             -- flip to going up
144         else
145             breath_count <= breath_count - 1;
146         end if;
147     end if;
148 end if;

149
150     nolurcalis <= 7 - to_integer(breath_count(6
151         downto 4));
152
153     end case;
154 end if;
155 end if;
156 end process;
157
158 end Behavioral;

```

Listing 11: beat_color_auto.vhd

A.4 moving_average.vhd

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity moving_average is
6      Port (
7          clk          : in  STD_LOGIC;
8          reset        : in  STD_LOGIC;
9          sample_valid  : in  STD_LOGIC;           -- pulse when
10             new sample ready
11          audio_in      : in  STD_LOGIC_VECTOR(11 downto 0); -- raw XADC
12             audio
13          audio_filtered : out STD_LOGIC_VECTOR(11 downto 0) -- smoothed
14             audio
15      );
16 end moving_average;
17
18 architecture Behavioral of moving_average is
19
20     -- Number of samples to average.
21     constant N : integer := 4;
22
23     type sample_array is array (0 to N-1) of unsigned(11 downto 0);

```

```

21     signal samples : sample_array := (others => (others => '0'));
22
23     -- Sum of 4 samples. Max value = 4 * 4095 = 16380, fits in 14 bits.
24     signal sum : unsigned(13 downto 0);
25
26 begin
27     process(clk)
28     begin
29         if rising_edge(clk) then
30             if reset = '1' then
31                 -- Clear all stored samples
32                 samples <= (others => (others => '0'));
33             elsif sample_valid = '1' then
34                 -- Shift: samples(0) is newest, samples(3) is oldest
35                 samples(3) <= samples(2);
36                 samples(2) <= samples(1);
37                 samples(1) <= samples(0);
38                 samples(0) <= unsigned(audio_in);
39             end if;
40             -- If sample_valid is '0', samples hold their previous values
41         end if;
42     end process;
43
44     sum <= resize(samples(0), 14) +
45           resize(samples(1), 14) +
46           resize(samples(2), 14) +
47           resize(samples(3), 14);
48
49     audio_filtered <= std_logic_vector(sum(13 downto 2));
50
51 end Behavioral;

```

Listing 12: moving_average.vhd

A.5 hysteresis_detector.vhd

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity hysteresis_detector is
6      Port (
7          clk          : in  STD_LOGIC;
8          reset        : in  STD_LOGIC;
9          audio_filtered : in  STD_LOGIC_VECTOR(11 downto 0); -- from
              moving_average
10         thresh_high   : in  STD_LOGIC_VECTOR(11 downto 0); -- upper
              threshold
11         thresh_low    : in  STD_LOGIC_VECTOR(11 downto 0); -- lower
              threshold
12         beat_detected  : out STD_LOGIC                       -- single-cycle

```

```

13         pulse
14     );
15 end hysteresis_detector;
16
17 architecture Behavioral of hysteresis_detector is
18
19     -- FSM state type
20     type state_type is (ARMED, FIRED);
21     signal state : state_type := ARMED;
22
23     -- Internal beat pulse signal
24     signal beat_pulse : std_logic := '0';
25
26 begin
27
28     process(clk)
29     begin
30         if rising_edge(clk) then
31             if reset = '1' then
32                 state      <= ARMED;
33                 beat_pulse <= '0';
34
35             else
36                 -- DEFAULT: beat_pulse is low every cycle unless explicitly
37                 -- set.
38                 -- This is what makes beat_pulse a single-cycle pulse instead
39                 -- of a latching signal.
40                 beat_pulse <= '0';
41
42                 case state is
43
44                     when ARMED =>
45                         -- Waiting for a beat. Check if signal crossed above
46                         -- THRESH_HIGH.
47                         if unsigned(audio_filtered) > unsigned(thresh_high)
48                         then
49                             beat_pulse <= '1';
50                             -- fire the pulse THIS cycle
51                             state      <= FIRED;
52                             -- lock out further beats
53                         end if;
54
55                     when FIRED =>
56                         -- Beat already fired. Wait for signal to drop below
57                         -- THRESH_LOW.
58                         if unsigned(audio_filtered) < unsigned(thresh_low)
59                         then
60                             state <= ARMED;
61                             -- re-arm the detector
62                         end if;
63
64                     end case;
65
66                 end case;
67
68             end if;
69         end process;
70
71 end Behavioral;

```

```

59         end if;
60     end if;
61 end process;
62
63     beat_detected <= beat_pulse;
64
65 end Behavioral;

```

Listing 13: hysteresis_detector.vhd

A.6 manual_color_select.vhd

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity manual_color_select is
5      Port (
6          -- Physical switch inputs
7          color_sel      : in  STD_LOGIC_VECTOR(3 downto 0);
8          brightness     : in  STD_LOGIC_VECTOR(2 downto 0);
9          led_enable     : in  STD_LOGIC;
10
11          -- Outputs to color_lut and driver
12          color_index    : out STD_LOGIC_VECTOR(3 downto 0);
13          brightness_level : out STD_LOGIC_VECTOR(2 downto 0);
14          enable         : out STD_LOGIC
15      );
16 end manual_color_select;
17
18 architecture Behavioral of manual_color_select is
19 begin
20
21     color_index      <= color_sel;
22     brightness_level <= brightness;
23     enable          <= led_enable;
24
25 end Behavioral;

```

Listing 14: manual_color_select.vhd

A.7 binary_input_to_7_segment_display.vhd

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity binary_input_to_7_segment_display is
5      Port ( bcd_in : in  STD_LOGIC_VECTOR (3 downto 0);
6            seg_out : out STD_LOGIC_VECTOR (6 downto 0));
7  end binary_input_to_7_segment_display;

```

```

8
9  architecture Behavioral of binary_input_to_7_segment_display is
10
11  begin
12  process(bcd_in)
13      begin
14          case bcd_in is
15              when "0000" => seg_out <= "1000000"; -- '0'
16              when "0001" => seg_out <= "1111001"; -- '1'
17              when "0010" => seg_out <= "0100100"; -- '2'
18              when "0011" => seg_out <= "0110000"; -- '3'
19              when "0100" => seg_out <= "0011001"; -- '4'
20              when "0101" => seg_out <= "0010010"; -- '5'
21              when "0110" => seg_out <= "0000010"; -- '6'
22              when "0111" => seg_out <= "1111000"; -- '7'
23              when "1000" => seg_out <= "0000000"; -- '8'
24              when "1001" => seg_out <= "0010000"; -- '9'
25
26              -- Hex letters for inputs 10 to 15
27              when "1010" => seg_out <= "0001000"; -- 'A'
28              when "1011" => seg_out <= "0000011"; -- 'b'
29              when "1100" => seg_out <= "1000110"; -- 'C'
30              when "1101" => seg_out <= "0100001"; -- 'd'
31              when "1110" => seg_out <= "0000110"; -- 'E'
32              when "1111" => seg_out <= "0001110"; -- 'F'
33
34              when others => seg_out <= "1111111"; -- Keep all off as a
35                  failsafe
36          end case;
37      end process;
38  end Behavioral;

```

Listing 15: binary_input_to_7_segment_display.vhd

A.8 anode_decoder.vhd

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity anode_decoder is
5      Port (
6          digit_select : in  STD_LOGIC_VECTOR (1 downto 0); -- The 2-bit clock
7                          signal
8          an            : out STD_LOGIC_VECTOR (3 downto 0)  -- The 4 screen
9                          power wires
10      );
11  end anode_decoder;
12
13  architecture Behavioral of anode_decoder is
14  begin

```

```

13     process(digit_select)
14     begin
15         case digit_select is
16             when "00" => an <= "1110"; -- Turns ON Digit 0 (Rightmost)
17             when "01" => an <= "1101"; -- Turns ON Digit 1
18             when "10" => an <= "1011"; -- Turns ON Digit 2
19             when "11" => an <= "0111"; -- Turns ON Digit 3 (Leftmost)
20             when others => an <= "1111"; -- Keeps all OFF as a failsafe
21         end case;
22     end process;
23 end Behavioral;

```

Listing 16: anode_decoder.vhd

A.9 clock_divider.vhd

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_UNSIGNED.ALL;
4
5  entity clock_divider is
6      Port ( clk : in STD_LOGIC;
7            digit_select : out STD_LOGIC_VECTOR (1 downto 0));
8  end clock_divider;
9
10 architecture Behavioral of clock_divider is
11
12     -- The 20-bit counter
13     signal refresh_counter : STD_LOGIC_VECTOR (19 downto 0) := (others =>
14         '0');
15
16 begin
17     process(clk)
18     begin
19         if rising_edge(clk) then
20             refresh_counter <= refresh_counter + 1;
21         end if;
22     end process;
23
24     digit_select <= refresh_counter(19 downto 18);
25 end Behavioral;

```

Listing 17: clock_divider.vhd

A.10 mux_4_to_1.vhd

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;

```

```

3
4  entity mux_4to1 is
5      Port (
6          sw          : in  STD_LOGIC_VECTOR (15 downto 0); -- All 16 bits
7          digit_select : in  STD_LOGIC_VECTOR (1 downto 0); -- The 2-bit
                        clock signal
8          digit_out    : out STD_LOGIC_VECTOR (3 downto 0)   -- The chosen 4
                        switches
9      );
10 end mux_4to1;
11
12 architecture Behavioral of mux_4to1 is
13 begin
14     process(digit_select, sw)
15     begin
16         case digit_select is
17             when "00" => digit_out <= sw(3 downto 0); -- Group 1
18             when "01" => digit_out <= sw(7 downto 4); -- Group 2
19             when "10" => digit_out <= sw(11 downto 8); -- Group 3
20             when "11" => digit_out <= sw(15 downto 12); -- Group 4
21             when others => digit_out <= "0000"; -- Failsafe
22         end case;
23     end process;
24 end Behavioral;

```

Listing 18: mux_4_to_1.vhd

Appendix B. XDC Constraints File

The full constraints file mapping the top-level block-design ports to the physical Basys 3 pins.

B.1 constraints.xdc

```

1  set_property PACKAGE_PIN W5 [get_ports clk_0_0]
2  set_property IOSTANDARD LVCMOS33 [get_ports clk_0_0]
3  create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports
   clk_0_0]
4
5  set_property PACKAGE_PIN U18 [get_ports reset_0_0]
6  set_property IOSTANDARD LVCMOS33 [get_ports reset_0_0]
7
8  set_property PACKAGE_PIN J1 [get_ports led_data_out_0_0]
9  set_property IOSTANDARD LVCMOS33 [get_ports led_data_out_0_0]
10
11 set_property PACKAGE_PIN V17 [get_ports {thres_high_sw[0]}]
12 set_property IOSTANDARD LVCMOS33 [get_ports {thres_high_sw[0]}]
13
14 set_property PACKAGE_PIN V16 [get_ports {thres_high_sw[1]}]
15 set_property IOSTANDARD LVCMOS33 [get_ports {thres_high_sw[1]}]
16

```

```

17 set_property PACKAGE_PIN W16 [get_ports {thres_high_sw[2]}]
18 set_property IOSTANDARD LVCMOS33 [get_ports {thres_high_sw[2]}]
19
20 set_property PACKAGE_PIN W17 [get_ports {thres_high_sw[3]}]
21 set_property IOSTANDARD LVCMOS33 [get_ports {thres_high_sw[3]}]
22
23 set_property PACKAGE_PIN W15 [get_ports {thres_high_sw[4]}]
24 set_property IOSTANDARD LVCMOS33 [get_ports {thres_high_sw[4]}]
25
26 set_property PACKAGE_PIN V15 [get_ports {thres_high_sw[5]}]
27 set_property IOSTANDARD LVCMOS33 [get_ports {thres_high_sw[5]}]
28
29 set_property PACKAGE_PIN W14 [get_ports {thres_low_sw[0]}]
30 set_property IOSTANDARD LVCMOS33 [get_ports {thres_low_sw[0]}]
31
32 set_property PACKAGE_PIN W13 [get_ports {thres_low_sw[1]}]
33 set_property IOSTANDARD LVCMOS33 [get_ports {thres_low_sw[1]}]
34
35 set_property PACKAGE_PIN V2 [get_ports {thres_low_sw[2]}]
36 set_property IOSTANDARD LVCMOS33 [get_ports {thres_low_sw[2]}]
37
38 set_property PACKAGE_PIN T3 [get_ports {thres_low_sw[3]}]
39 set_property IOSTANDARD LVCMOS33 [get_ports {thres_low_sw[3]}]
40
41 set_property PACKAGE_PIN T2 [get_ports {thres_low_sw[4]}]
42 set_property IOSTANDARD LVCMOS33 [get_ports {thres_low_sw[4]}]
43
44 set_property PACKAGE_PIN R3 [get_ports {thres_low_sw[5]}]
45 set_property IOSTANDARD LVCMOS33 [get_ports {thres_low_sw[5]}]
46
47 set_property PACKAGE_PIN R2 [get_ports led_enable_0_0]
48 set_property IOSTANDARD LVCMOS33 [get_ports led_enable_0_0]
49
50 set_property PACKAGE_PIN W7 [get_ports {seg_out_0_0[0]}]
51 set_property IOSTANDARD LVCMOS33 [get_ports {seg_out_0_0[0]}]
52
53 set_property PACKAGE_PIN W6 [get_ports {seg_out_0_0[1]}]
54 set_property IOSTANDARD LVCMOS33 [get_ports {seg_out_0_0[1]}]
55
56 set_property PACKAGE_PIN U8 [get_ports {seg_out_0_0[2]}]
57 set_property IOSTANDARD LVCMOS33 [get_ports {seg_out_0_0[2]}]
58
59 set_property PACKAGE_PIN V8 [get_ports {seg_out_0_0[3]}]
60 set_property IOSTANDARD LVCMOS33 [get_ports {seg_out_0_0[3]}]
61
62 set_property PACKAGE_PIN U5 [get_ports {seg_out_0_0[4]}]
63 set_property IOSTANDARD LVCMOS33 [get_ports {seg_out_0_0[4]}]
64
65 set_property PACKAGE_PIN V5 [get_ports {seg_out_0_0[5]}]
66 set_property IOSTANDARD LVCMOS33 [get_ports {seg_out_0_0[5]}]
67
68 set_property PACKAGE_PIN U7 [get_ports {seg_out_0_0[6]}]

```

```
69 set_property IOSTANDARD LVCMOS33 [get_ports {seg_out_0_0[6]}]
70
71 set_property PACKAGE_PIN U2 [get_ports {an_0_0[0]}]
72 set_property IOSTANDARD LVCMOS33 [get_ports {an_0_0[0]}]
73
74 set_property PACKAGE_PIN U4 [get_ports {an_0_0[1]}]
75 set_property IOSTANDARD LVCMOS33 [get_ports {an_0_0[1]}]
76
77 set_property PACKAGE_PIN V4 [get_ports {an_0_0[2]}]
78 set_property IOSTANDARD LVCMOS33 [get_ports {an_0_0[2]}]
79
80 set_property PACKAGE_PIN W4 [get_ports {an_0_0[3]}]
81 set_property IOSTANDARD LVCMOS33 [get_ports {an_0_0[3]}]
82
83 set_property PACKAGE_PIN A14 [get_ports vauxp6_0_0]
84 set_property IOSTANDARD LVCMOS33 [get_ports vauxp6_0_0]
85
86 set_property PACKAGE_PIN A13 [get_ports vauxn6_0_0]
87 set_property IOSTANDARD LVCMOS33 [get_ports vauxn6_0_0]
```

Listing 19: constraints.xdc

Appendix C. Vivado Block Design Screenshots

The following figures provide high-resolution exports of the three primary Vivado Block Designs used in this project: the LED DSP subsystem, the seven-segment display subsystem, and the top-level master wrapper.

C.1 seven_segment_display Block Design

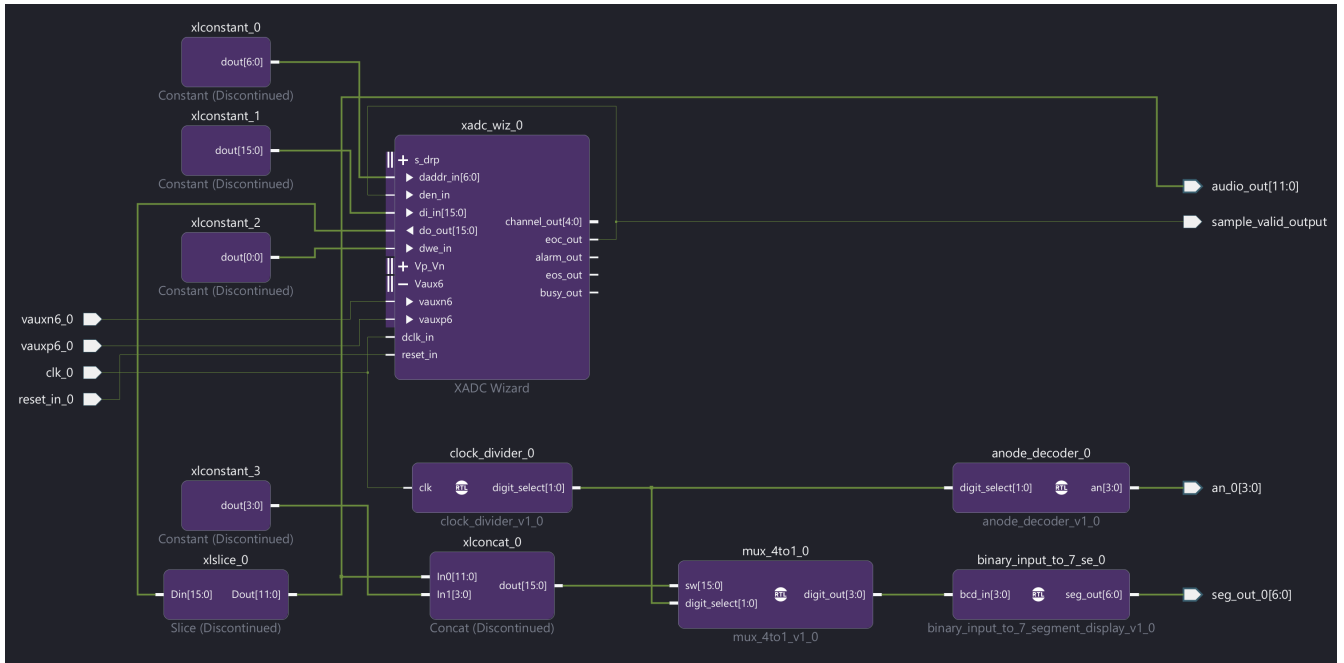


Figure 13: Detailed high-resolution export of the `seven_segment_display` Block Design.

C.2 led_subsystem Block Design

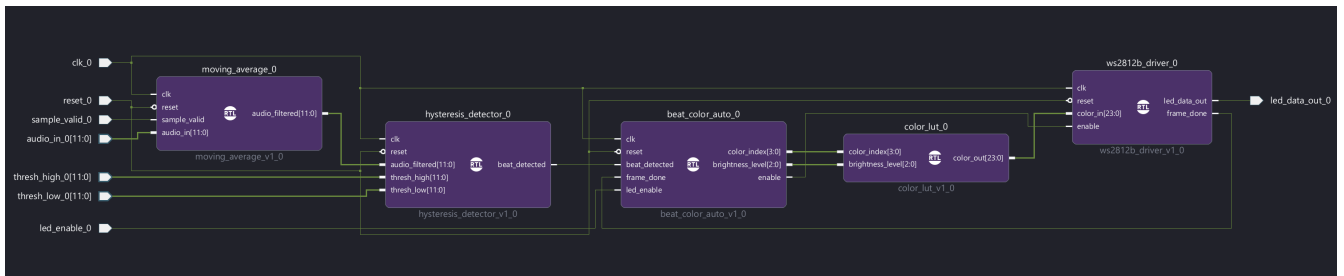


Figure 14: Detailed high-resolution export of the `led_subsystem` Block Design.

C.3 master_wrapper Top-Level Block Design

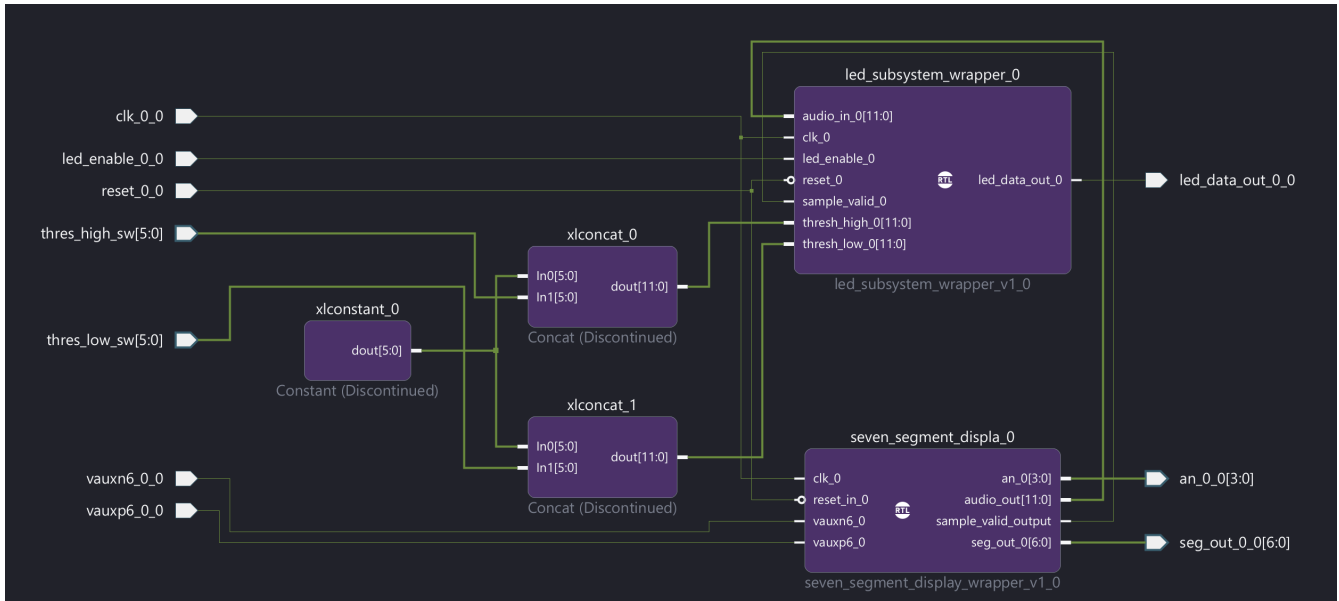


Figure 15: Detailed high-resolution export of the `master_wrapper` Top-Level Block Design.

Appendix D. Video Link

Youtube video link: <https://youtu.be/OFfqUWP0drM>